

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

П РА К Т И Ч Е С К А Я Р А Б О Т А № 2

Разработка кода через тестирование поведения
по дисциплине «Методы тестирования программного обеспечения»

Выполнил
студент гр. з5130903/40002

И.Р. Фёдорова

Руководитель
старший преподаватель ВШ ПИ

В.А. Пархоменко

СОДЕРЖАНИЕ

Введение	4
1 Проектирование консольного приложения	5
1.1 Описание решаемой задачи	5
1.2 Конкурирующие способы решения задачи	6
1.3 Алгоритм Манакера	7
1.4 Варианты использования приложения.....	8
1.5 Функциональность приложения	9
1.6 Формат входных и выходных данных.....	10
1.7 Рекомендательная информация для пользователя	11
1.8 Структура приложения	11
1.9 Выводы	13
2 Проектирование тестов	13
2.1 Проектирование тестов на основе выбранных технологий.....	13
2.1.1 Метод разбиения на классы эквивалентности	14
2.1.2 Метод анализа граничных значений.....	16
2.1.3 Тестирование операторов.....	18
2.1.4 Тестирование ветвей	20
2.2 Сравнительный анализ методов проектирования тестов.....	21
2.3 Проектирование тестов для загрузки строки и вывода результата	22
2.4 Выводы	23
3 Описание библиотек для модульных тестов	24
3.1 Jest	24
3.2 Sinon.....	24
3.3 Chai.....	25
3.4 Assumption-style проверки	25
3.5 Сводная таблица используемых средств	26
3.6 Выводы	26
4 Реализация модульных тестов	26
4.1 Структура тестового набора.....	27
4.2 Параметризованные тесты.....	27
4.3 Реализация assertion и matchers.....	28
4.4 Assumption-style проверки и test doubles	28
4.5 Характерные тестовые сценарии	28
4.6 Результаты запуска тестов	29

4.7 Выводы	29
5 Реализация консольного приложения	30
5.1 Структура проекта	30
5.2 Описание основных модулей	30
5.3 Описание реализации вычислительной части	31
5.4 Тестирование	32
5.5 Демонстрация работы приложения	32
5.6 Выводы	34
6 Мутационное тестирование	35
6.1 Назначение мутационного тестирования	35
6.2 Настройка StrykerJS	35
6.3 Используемые мутационные операторы	36
6.4 Полученные метрики	36
6.5 Подробный пример анализа мутантов	37
6.6 Эквивалентные мутации	37
6.7 Оценка mutation-adequate	38
6.8 Выводы	38
Заключение	39
Список сокращений и условных обозначений	41
Список использованных источников	42
Приложение 1 Полный листинг кода приложения	43
Приложение 2 Полный листинг кода тестов	55

ВВЕДЕНИЕ

Поведенческая разработка через тестирование (Behaviour-Driven Development, BDD) представляет собой современную методологию разработки программного обеспечения, объединяющую процессы проектирования, реализации и проверки качества программных решений. В условиях повышения сложности программных систем и необходимости эффективного взаимодействия между разработчиками, тестировщиками и заказчиками BDD выступает важным инструментом формализации требований посредством исполняемых спецификаций, описанных на естественном языке.

Задачи практической работы:

- А. изучить синтаксис языка Gherkin и его основные конструкции (Feature, Rule, Background, Scenario, Scenario Outline);
- В. разработать пользовательские истории (User Story) для функциональных возможностей приложения, реализующего поиск палиндромных подстрок;
- С. реализовать BDD-сценарии в декларативном и императивном стилях;
- Д. реализовать Step Definitions на языке JavaScript с использованием фреймворка Cucumber;
- Е. настроить инфраструктуру запуска BDD-тестов в JavaScript-проекте и генерацию отчётов о результатах тестирования.

Практическая часть работы посвящена разработке набора BDD-тестов для приложения, реализующего алгоритм Манакера, предназначенный для эффективного поиска самой длинной палиндромной подстроки в строке за линейное время.

1 1 BDD-ТЕСТИРОВАНИЕ

В данной главе описано планирование и реализация BDD-тестирования (Behaviour-Driven Development) приложения алгоритма Манакера. Описывается методология BDD, синтаксис языка Gherkin, выбор инструментария, а также разработанные тестовые сценарии для трёх ключевых функциональных областей системы.

1.1 Описание решаемой задачи

Пусть дана строка $s = s_0s_1 \dots s_{n-1}$ длины n . Требуется определить:

- А. массив радиусов нечётных палиндромов с центрами в каждой позиции строки;
- В. массив радиусов чётных палиндромов с центрами между соседними символами;
- С. наибольшую палиндромную подстроку;
- Д. количество палиндромных подстрок;
- Е. дополнительные характеристики строки, полезные пользователю при анализе результата.

Под палиндромной подстрокой понимается непрерывный фрагмент исходной строки, читающийся одинаково слева направо и справа налево. Для нечётного палиндрома центр совпадает с символом строки, а для чётного располагается между двумя соседними символами. Задача поиска палиндромных подстрок относится к задачам обработки строк и находит применение при анализе текстов, биологических последовательностей и повторяющихся симметричных шаблонов [gusfield1997].

Наивный способ решения заключается в рассмотрении каждого возможного центра и последовательном расширении границ палиндрома. Такой подход имеет асимптотику $O(n^2)$ в худшем случае. Для длинных строк это приводит к избыточным затратам времени. Алгоритм Манакера позволяет вычислить массивы радиусов всех палиндромов за линейное время $O(n)$, используя информацию о ранее найденных палиндромах [manacher1975].

1.2 Конкурирующие способы решения задачи

В соответствии с заданием на практическую работу приложение должно содержать не менее двух различных способов решения основной задачи. Для проекта выбраны два алгоритма:

- А. алгоритм расширения от центра;
- В. алгоритм Манакера.

Первый алгоритм используется как базовый эталон для сравнения результатов при тестировании. Он последовательно рассматривает все возможные центры палиндромов и расширяет границы, пока символы совпадают. В худшем случае его трудоёмкость составляет $O(n^2)$, однако он прост по структуре и удобен как проверочный алгоритм.

Второй алгоритм – алгоритм Манакера – является основным производственным решением. Он выполняется отдельно для нечётных и чётных палиндромов, поддерживая текущий интервал $[l, r]$ с максимальной правой границей и зеркальную позицию относительно центра. Благодаря этому алгоритм избегает повторной работы и обеспечивает линейную сложность.

Сравнение двух подходов приведено в табл.1.1.

Таблица 1.1

Сравнение двух способов поиска палиндромных подстрок

Характеристика	Расширение от центра	Алгоритм Манакера
Идея	Независимая обработка каждого центра	Использование уже найденного максимального интервала $[l, r]$ и зеркальной позиции
Сложность	$O(n^2)$	$O(n)$
Простота реализации	Очень высокая	Средняя
Удобство как эталона для тестов	Высокое	Среднее
Эффективность на длинных строках	Низкая	Высокая
Вычисление всех радиусов	Да	Да

1.3 Алгоритм Манакера

Алгоритм Манакера выполняется отдельно для массивов d_{odd} и d_{even} . Для нечётного случая $d_{odd}[i]$ обозначает радиус палиндрома с центром в позиции i , включая центральный символ. Для чётного случая $d_{even}[i]$ соответствует палиндрому с центром между символами $i - 1$ и i .

Общая идея алгоритма включает следующие шаги:

- А. инициализировать границы l и r текущего палиндрома с максимальной правой границей;
- В. для каждой позиции определить стартовый радиус: если позиция находится вне интервала $[l, r]$, радиус задаётся начальным значением; если позиция попадает внутрь интервала, используется значение симметричной позиции;
- С. расширять палиндром, пока символы по обе стороны совпадают;
- Д. при необходимости обновить текущие границы l и r ;
- Е. после завершения обработки всех позиций использовать массивы радиусов для восстановления наибольшего палиндрома и подсчёта числа палиндромных подстрок.

На рис.1.1 представлена блок-схема вычисления радиусов палиндромов алгоритмом Манакера.

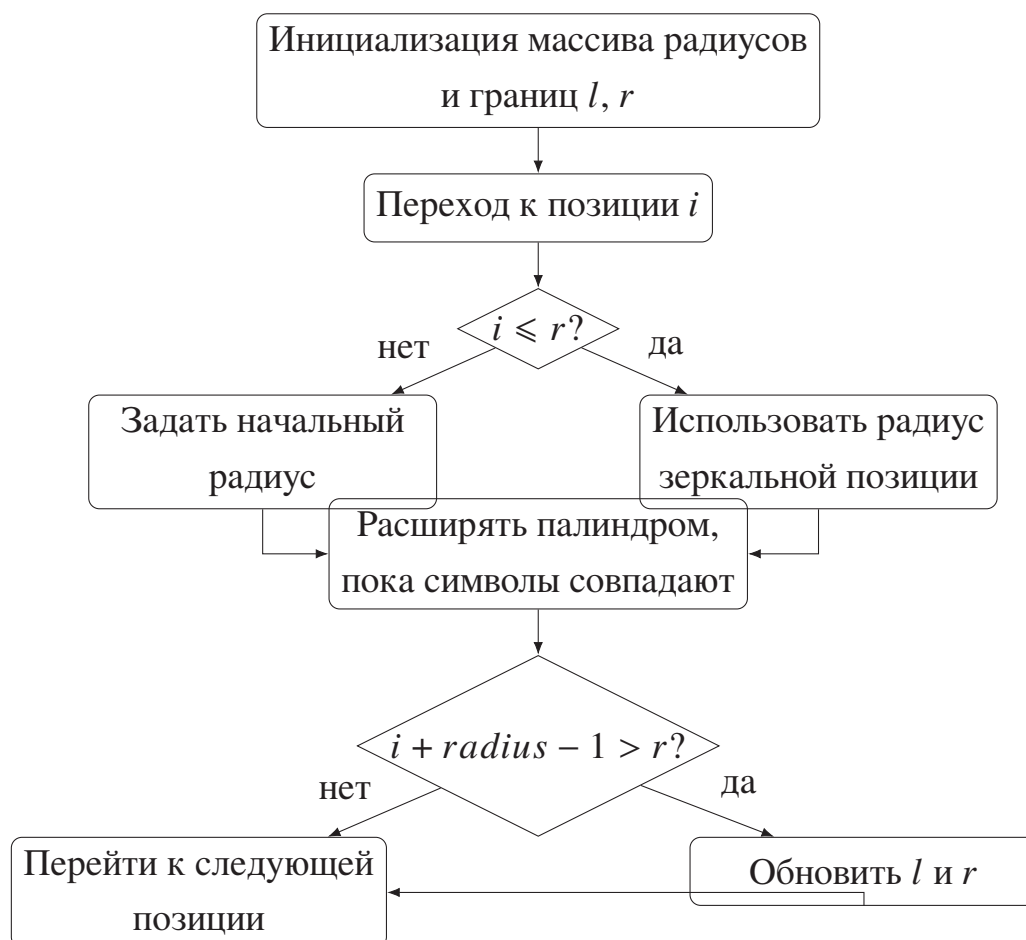


Рис.1.1. Блок-схема вычисления радиусов палиндромов алгоритмом Манакера

1.4 Варианты использования приложения

В системе определён один актор – пользователь. Он взаимодействует с приложением через консольный интерфейс и может вводить строку вручную либо загружать её из JSON-файла. После этого пользователь получает отчёт о палиндромных подстроках, сохраняет результаты и анализирует рекомендационную информацию.

На рис.1.2 представлена use-case диаграмма приложения.



Рис.1.2. Диаграмма вариантов использования консольного приложения

1.5 Функциональность приложения

После запуска приложение отображает приветственный баннер, текущее состояние загруженных данных и главное меню. В шапке меню выводится краткая информация: введена ли строка, какова её длина и был ли ранее сформирован результат анализа. Структура главного меню приведена в табл.1.2.

Таблица 1.2

Структура и функциональность главного меню

Пункт	Название	Описание
1	Ввести строку вручную	Пользователь вводит строку с клавиатуры; предыдущий результат сбрасывается
2	Загрузить строку из JSON-файла	Потоково считывается JSON-файл произвольного размера, содержащий строку для анализа

Продолжение табл. 1.2

Пункт	Название	Описание
3	Запустить алгоритм Манакера	Вычисляются d_{odd} , d_{even} , наибольший палиндром и число палиндромов
4	Запустить расширение от центра	Формируется эталонный результат для сравнения и верификации
5	Показать подробный отчёт	Выводятся массивы радиусов, длина наибольшего палиндрома и статистика
6	Сохранить результат в JSON-файл	Последний результат анализа сохраняется в файл
7	Справка	Отображается описание форматов данных и пояснения к результату
0	Выход	Завершение работы приложения

Справка приложения содержит:

- описание формата входного JSON-файла;
- инструкцию по ручному вводу строки;
- пояснение значения массивов d_{odd} и d_{even} ;
- описание структуры JSON-файла результата;
- рекомендации по интерпретации полученных характеристик строки.

1.6 Формат входных и выходных данных

Входной JSON-файл содержит исходную строку и, при необходимости, дополнительные настройки анализа. Используется компактный формат:

```

1 | {
2 |   "input": "abacaba",
3 |   "options": {
4 |     "ignoreCase": false
5 |   }
6 | }
```

Рис.1.3. Пример входного JSON-файла

При ручном вводе пользователь вводит строку целиком одной строкой. Строка может содержать пробелы, цифры и специальные символы.

Выходной JSON-файл содержит имя использованного алгоритма, исходную строку, массивы радиусов и агрегированный результат анализа. Пример выходного формата приведён на рис.1.4.

```

1 {
2   "algorithm": "manacher",
3   "input": "abacaba",
4   "oddRadii": [1, 2, 1, 4, 1, 2, 1],
5   "evenRadii": [0, 0, 0, 0, 0, 0, 0],
6   "longestPalindrome": {
7     "value": "abacaba",
8     "start": 0,
9     "end": 6,
10    "length": 7
11  },
12  "palindromeCount": 12,
13  "recommendation": "Строка целиком является палиндромом.",
14  "comparison": {
15    "referenceAlgorithm": "center-expansion",
16    "matches": true
17  }
18 }

```

Рис.1.4. Пример выходного JSON-файла

1.7 Рекомендательная информация для пользователя

Согласно заданию приложение должно формировать полезную рекомендательную информацию. В проекте предусмотрены следующие типы рекомендаций:

- А. если наибольший палиндром совпадает со всей строкой, выводится сообщение, что строка целиком симметрична;
- В. если длина наибольшего палиндрома составляет значительную долю строки, приложение сообщает о высокой палиндромной плотности;
- С. если в строке преобладают только короткие палиндромы длины 1–2, пользователь получает сообщение о слабой симметричности структуры;
- Д. если длинный глобальный палиндром отсутствует, приложение сообщает о наличии нескольких локальных палиндромных областей.

Наличие таких сообщений повышает прикладную ценность программы и делает результат полезнее, чем простой вывод массивов радиусов.

1.8 Структура приложения

Проект реализуется на JavaScript в среде Node.js. При проектировании выделены следующие модули:

- `src/analyzers/manacherOdd.js` – вычисление нечётных радиусов;
- `src/analyzers/manacherEven.js` – вычисление чётных радиусов;
- `src/analyzers/centerExpansion.js` – эталонный алгоритм расширения от центра;
- `src/services/palindromeService.js` – агрегирование результатов;
- `src/io/jsonReader.js` – загрузка входного JSON;
- `src/io/resultWriter.js` – вывод отчёта в консоль и файл;
- `src/ui/consoleUi.js` – консольное меню;
- `src/index.js` – точка входа.

Такое разбиение позволяет независимо тестировать вычислительную логику, операции ввода-вывода и пользовательский интерфейс. На рис.1.5 представлена блок-схема общего сценария работы приложения.



Рис.1.5. Блок-схема общего сценария работы приложения

1.9 Выводы

В данной главе выполнено проектирование консольного приложения для анализа палиндромных подстрок. Определены два конкурирующих способа решения задачи: расширение от центра и алгоритм Манакера. Описаны варианты использования, структура меню, форматы входных и выходных JSON-данных, а также механизм формирования рекомендаций для пользователя. Полученная архитектура удовлетворяет требованиям задания и служит основой для последующего проектирования тестов и реализации программы.

2 ПРОЕКТИРОВАНИЕ ТЕСТОВ

В данной главе описаны техники проектирования тестов, применённые в работе. Для каждой техники сформированы проверяемые условия (test cover items), контрольные примеры (test cases) и оптимизированные наборы тестов. Дополнительно приведено сравнение выбранных методов по покрытию, трудоёмкости и практической ценности.

2.1 Проектирование тестов на основе выбранных технологий

Стандарт ISO/IEC/IEEE 29119-4:2021 выделяет три группы техник проектирования тестов: методы на основе спецификации, методы на основе структуры тестируемого объекта и методы на основе опыта тестировщика [iso29119-4]. В данной работе сознательно используются первые две группы. Методы на основе спецификации применяются к требованиям, форматам входных данных и ожидаемым результатам, то есть не требуют знания исходного кода. Методы на основе структуры применяются к исходному коду и позволяют проверить операторы, решения, ветви и другие элементы управляющего потока.

Для проектирования тестов приложения выбраны следующие техники на основе спецификации:

- разбиение на классы эквивалентности (Equivalence Partitioning, EP);
- анализ граничных значений (Boundary Value Analysis, BVA).

Из техник на основе структуры выбраны:

- тестирование операторов (Statement Testing);

– тестирование ветвей (Branch Testing).

Таким образом, EP и BVA используются как техники класса specification-based, соответствующие группе техник из приложения В стандарта, а Statement Testing и Branch Testing используются как техники класса structure-based, соответствующие группе техник из приложения С. Такое разделение важно для трассируемости: входные и выходные классы проектируются по требованиям к приложению, а внутренние условия алгоритма Манакера проверяются структурными тестами по коду.

2.1.1 Метод разбиения на классы эквивалентности

Метод EP в ISO/IEC/IEEE 29119-4:2021 относится к техникам на основе спецификации и предполагает разбиение входов или выходов тестируемого объекта на классы, которые должны обрабатываться одинаково. В данной работе EP применяется к пользовательским входам, JSON-вводу и наблюдаемым результатам анализа строки. Для алгоритма Манакера такими классами являются строки разной длины и различные семантические типы строк: полностью палиндромные, непалиндромные, содержащие чётные или нечётные максимальные палиндромы, строки с повторяющимися символами и т.д. Внутренние состояния алгоритма, например нахождение позиции внутри интервала $[l, r]$, не рассматриваются как самостоятельные классы эквивалентности; они используются далее в структурных техниках Statement Testing и Branch Testing.

Проверяемые условия для метода EP приведены в табл.2.1.

Таблица 2.1

Проверяемые условия с использованием метода EP

Имя проверки	Содержание проверки	Слой данных	Результат
EP-TCI-1	Строка пустая	Входные	0
EP-TCI-2	Строка длины 1	Входные	1
EP-TCI-3	Строка длины 2 без чётного палиндрома	Входные	1
EP-TCI-4	Строка длины 2 с чётным палиндромом	Входные	1
EP-TCI-5	Строка без палиндромов длины > 1	Входные	1
EP-TCI-6	Строка, целиком являющаяся нечётным палиндромом	Входные	1
EP-TCI-7	Строка, целиком являющаяся чётным палиндромом	Входные	1

Продолжение табл. 2.1

Имя проверки	Содержание проверки	Слой данных	Результат
EP-TCI-8	Строка с несколькими перекрывающимися нечётными палиндромами	Входные	1
EP-TCI-9	Строка с несколькими перекрывающимися чётными палиндромами	Входные	1
EP-TCI-10	Строка из одинаковых символов	Входные	1
EP-TCI-11	Строка с пробелами и специальными символами	Входные	1
EP-TCI-12	Корректный JSON-файл содержит поле <code>input</code>	Входные	1
EP-TCI-13	JSON-файл не содержит обязательного поля <code>input</code>	Входные	0
EP-TCI-14	Файл по пути отсутствует	Входные	0
EP-TCI-15	Наибольший палиндром имеет нечётную длину	Выходные	1
EP-TCI-16	Наибольший палиндром имеет чётную длину	Выходные	1
EP-TCI-17	Все чётные радиусы равны 0	Выходные	1
EP-TCI-18	Присутствуют ненулевые чётные радиусы	Выходные	1
EP-TCI-19	Формируется рекомендация о полной симметрии строки	Выходные	1
EP-TCI-20	Формируется рекомендация о низкой палиндромной плотности	Выходные	1
EP-TCI-21	Результаты двух алгоритмов совпадают	Выходные	1
EP-TCI-22	Результаты двух алгоритмов расходятся	Выходные	0
EP-TCI-23	Результат успешно сохраняется в JSON-файл	Выходные	1

Контрольные примеры для метода EP приведены в табл.2.2.

Таблица 2.2

Контрольные примеры для метода EP

Test case	Входные данные	Ожидаемый вывод
EP-TC-1		Выброс исключения или сообщение о недопустимом вводе
EP-TC-2	"a"	$d_{odd} = [1]$, $d_{even} = [0]$, наибольший палиндром "a"
EP-TC-3	"ab"	$d_{odd} = [1,1]$, $d_{even} = [0,0]$
EP-TC-4	"aa"	наибольший палиндром "aa", чётный радиус в центре ненулевой
EP-TC-5	"abc"	все радиусы минимальны, рекомендация о слабой симметрии
EP-TC-6	"abacaba"	наибольший палиндром совпадает со всей строкой

Продолжение табл. 2.2

Test case	Входные данные	Ожидаемый вывод
EP-TC-7	"abba"	наибольший палиндром чётной длины 4
EP-TC-8	"ababa"	несколько перекрывающихся нечётных палиндромов
EP-TC-9	"abbaabba"	несколько крупных чётных палиндромов
EP-TC-10	"aaaaa"	возрастающие радиусы, максимальное число палиндромов
EP-TC-11	"a-b-a"	специальные символы обрабатываются как обычные
EP-TC-12	JSON с "input": "level"	корректное чтение строки из файла
EP-TC-13	JSON без поля input	ошибка валидации формата
EP-TC-14	несуществующий путь к файлу	ошибка чтения файла

В результате оптимизации 14 базовых тест-кейсов сокращены до 8 оптимизированных при сохранении покрытия существенных входных и выходных классов эквивалентности. Оптимизированный набор представлен в табл.2.3.

Таблица 2.3

Оптимизированные контрольные примеры для метода EP

Test case	Входные данные	Покрываемые EP-TCI	Ожидаемый вывод
EP-TC-1		1	Ошибка ввода
EP-TC-2	"a"	2, 15	минимальный корректный результат
EP-TC-3	"aa"	4, 16, 18	чётный палиндром длины 2
EP-TC-4	"abc"	3, 5, 17, 20	только одиночные палиндромы
EP-TC-5	"abacaba"	6, 8, 19, 21	полная симметрия строки
EP-TC-6	"abbaabba"	7, 9, 16, 18	крупные чётные палиндромы
EP-TC-7	"aaaaa"	10	максимальная палиндромная плотность
EP-TC-8	JSON / ошибка файла	11–14, 22, 23	проверка ввода-вывода

2.1.2 Метод анализа граничных значений

Метод BVA в ISO/IEC/IEEE 29119-4:2021 является техникой на основе спецификации и ориентирован на границы классов эквивалентности. В работе он

применяется прежде всего к границам входных и выходных доменов: минимальной допустимой длине строки, строке длины 2, пустому JSON-файлу, минимальной строке в JSON и крайним значениям длины найденного палиндрома. Границы индексов в алгоритме Манакера дополнительно используются как обоснование структурных тестов, поскольку они связаны с риском ошибок типа off-by-one в реализации.

Проверяемые условия для метода BVA приведены в табл.2.4.

Таблица 2.4

Проверяемые условия с использованием метода BVA

Имя проверки	Содержание проверки	Тип границы	Результат
BVA-TCI-1	Длина строки 0	Входные	0
BVA-TCI-2	Длина строки 1	Входные	1
BVA-TCI-3	Длина строки 2 без совпадения символов	Входные	1
BVA-TCI-4	Длина строки 2 с совпадением символов	Входные	1
BVA-TCI-5	Минимальный нечётный палиндром длины 3	Входные	1
BVA-TCI-6	Палиндром касается левой границы строки	Граница строки	1
BVA-TCI-7	Палиндром касается правой границы строки	Граница строки	1
BVA-TCI-8	Центр палиндрома находится в начале массива проверки	Алгоритмическая	1
BVA-TCI-9	Центр палиндрома находится в конце массива проверки	Алгоритмическая	1
BVA-TCI-10	Максимальный радиус обновляет правую границу ровно до $n - 1$	Алгоритмическая	1
BVA-TCI-11	Проверка условия выхода за левую границу даёт ложь на шаге -1	Алгоритмическая	1
BVA-TCI-12	Проверка условия выхода за правую границу даёт ложь на шаге n	Алгоритмическая	1
BVA-TCI-13	Пустой JSON-файл	Входные	0
BVA-TCI-14	JSON-файл с минимально допустимой строкой	Входные	1
BVA-TCI-15	Путь сохранения результата указывает на новый файл	Выходные	1
BVA-TCI-16	Наибольший палиндром длины 1	Выходные	1
BVA-TCI-17	Наибольший палиндром длины, равной длине всей строки	Выходные	1

Контрольные примеры BVA приведены в табл.2.5.

Таблица 2.5

Контрольные примеры для метода BVA

Test case	Входные данные	Ожидаемый вывод
BVA-TC-1		ошибка ввода
BVA-TC-2	"a"	минимально допустимый результат
BVA-TC-3	"ab"	два одиночных палиндрома
BVA-TC-4	"aa"	минимальный чётный палиндром
BVA-TC-5	"aba"	минимальный нечётный палиндром длины 3
BVA-TC-6	"baaab"	палиндром касается обеих границ строки
BVA-TC-7	"abccba"	правая граница обновляется до конца строки
BVA-TC-8	пустой JSON-файл	ошибка чтения
BVA-TC-9	JSON с "input": "x"	корректная загрузка минимальной строки
BVA-TC-10	сохранение в новый файл	успешная запись результата

Оптимизированный набор BVA-тестов приведён в табл.2.6.

Таблица 2.6

Оптимизированные контрольные примеры для метода BVA

Test case	Входные данные	Покрываемые BVA-TCI	Ожидаемый вывод
BVA-TC-1		1, 13	ошибка ввода
BVA-TC-2	"a"	2, 14, 16	минимальный корректный сценарий
BVA-TC-3	"aa"	3, 4	проверка длины 2
BVA-TC-4	"aba"	5, 8	нечётный палиндром минимальной длины
BVA-TC-5	"baaab"	6, 7, 10, 11, 12, 17	палиндром на границах строки
BVA-TC-6	"abccba"	9, 10	чётный палиндром на правой границе
BVA-TC-7	сохранение в новый файл	15	успешная запись результата

2.1.3 Тестирование операторов

Метод Statement Testing относится к техникам на основе структуры и ориентирован на то, чтобы каждый значимый исполняемый оператор программы

был выполнен хотя бы одним тестом. Для приложения поиска палиндромов это включает:

- инициализацию массивов радиусов;
- присваивание стартового радиуса;
- цикл расширения палиндрома;
- обновление границ l, r ;
- суммирование радиусов при подсчёте количества палиндромов;
- формирование JSON-результата.

Перечисленные условия являются структурными test coverage items: они не подменяют классы эквивалентности, а дополняют их анализом исполняемых операторов.

Проверяемые условия ST приведены в табл.2.7.

Таблица 2.7

Проверяемые условия с использованием Statement Testing

Имя проверки	Содержание проверки	Результат
ST-TCI-1	Инициализация массива d_{odd} выполнена	1
ST-TCI-2	Инициализация массива d_{even} выполнена	1
ST-TCI-3	Выполнено присваивание стартового радиуса вне интервала $[l, r]$	1
ST-TCI-4	Выполнено присваивание стартового радиуса по зеркальной позиции	1
ST-TCI-5	Выполнен хотя бы один шаг расширения палиндрома	1
ST-TCI-6	Выполнена запись нового значения радиуса в массив	1
ST-TCI-7	Выполнено обновление границ l, r	1
ST-TCI-8	Выполнен проход по всем позициям строки	1
ST-TCI-9	Выполнено восстановление нечётного максимального палиндрома	1
ST-TCI-10	Выполнено восстановление чётного максимального палиндрома	1
ST-TCI-11	Выполнено суммирование радиусов для подсчёта количества палиндромов	1
ST-TCI-12	Выполнено формирование рекомендации пользователю	1

Контрольные примеры ST приведены в табл.2.8.

Таблица 2.8

Контрольные примеры для Statement Testing

Test case	Входные данные	Ожидаемый вывод
ST-TC-1	"abacaba"	выполняются основные операторы алгоритма Манакера
ST-TC-2	"abba"	выполняется ветка чётного палиндрома и восстановление результата
ST-TC-3	сохранение результата в JSON	выполняется запись файла и формирование отчёта

2.1.4 Тестирование ветвей

Метод Branch Testing относится к техникам на основе структуры и ориентирован на покрытие ключевых ветвей условий в управляющем потоке программы. Для реализации алгоритма Манакера существенны следующие развилки:

- позиция находится внутри интервала $[l, r]$ / вне интервала;
- расширение возможно / невозможно;
- границы обновляются / не обновляются;
- максимальный палиндром чётный / нечётный;
- входной JSON корректен / некорректен.

Эти условия относятся к структуре управляющего потока и поэтому рассматриваются в рамках техник приложения С, а не как отдельные классы эквивалентности.

Проверяемые условия ВТ приведены в табл.2.9.

Таблица 2.9

Проверяемые условия с использованием Branch Testing

Имя проверки	Содержание проверки	Результат
ВТ-TCI-1	Ветвь обработки пустой строки	0
ВТ-TCI-2	Ветвь обработки корректной непустой строки	1
ВТ-TCI-3	Ветвь $i > r$	1
ВТ-TCI-4	Ветвь $i \leq r$	1
ВТ-TCI-5	Ветвь успешного расширения палиндрома	1
ВТ-TCI-6	Ветвь без расширения	1
ВТ-TCI-7	Ветвь обновления границ	1
ВТ-TCI-8	Ветвь без обновления границ	1
ВТ-TCI-9	Ветвь выбора нечётного максимального палиндрома	1
ВТ-TCI-10	Ветвь выбора чётного максимального палиндрома	1
ВТ-TCI-11	Ветвь корректной загрузки JSON	1

Продолжение табл. 2.9

Имя проверки	Содержание проверки	Результат
BT-TCI-12	Ветвь ошибки формата JSON	0

Контрольные примеры ВТ приведены в табл.2.10.

Таблица 2.10

Контрольные примеры для Branch Testing

Test case	Входные данные	Ожидаемый вывод
BT-TC-1		ошибка ввода
BT-TC-2	"abc"	покрытие ветви без расширения
BT-TC-3	"aaaa"	покрытие ветви с зеркальной позицией и расширением
BT-TC-4	"abba"	покрытие ветви выбора чётного максимального палиндрома
BT-TC-5	некорректный JSON	покрытие ветви ошибки формата

2.2 Сравнительный анализ методов проектирования тестов

Преимущества и недостатки выбранных методов приведены в табл.2.11.

Таблица 2.11

Преимущества и недостатки методов проектирования тестов

Метод	Преимущества	Недостатки
EP	Хорошо покрывает смысловые типы строк, сокращает число тестов за счёт выбора представителей классов, применим без готовой реализации	Может пропустить граничные дефекты, требует аккуратного выделения классов эквивалентности
BVA	Эффективно выявляет ошибки на границах длины и индексов, хорошо дополняет EP	Быстро увеличивает число тестов при росте числа параметров, хуже работает без чётких диапазонов
Statement Testing	Даёт минимальный структурный контроль, позволяет быстро проверить исполнение всех операторов	Не гарантирует покрытие альтернативных ветвей и исключительных путей
Branch Testing	Проверяет ключевые логические развилки, хорошо дополняет спецификационные методы	Требует знания реализации, не покрывает все возможные пути исполнения

Сводное сравнение покрытий, количества тестов и трудозатратности приведено в табл.2.12.

Таблица 2.12

Сравнение методов проектирования тестов по ключевым метрикам

Метрика	EP	BVA	Statement Testing	Branch Testing
Количество TCI	30	17	12	12
Количество ТС	14	10	3	5
Количество оптимизированных ТС	8	7	2–3	4–5
Трудозатратность проектирования	Высокая	Средняя	Низкая	Средняя
Требуется знания реализации	Нет	Нет	Частично	Да
Покрытие входных данных	Высокое	Высокое на границах	Низкое	Низкое
Покрытие кодовых путей	Низкое	Низкое	Базовое	Хорошее
Обнаружение граничных ошибок	Среднее	Высокое	Низкое	Среднее

EP обеспечивает наиболее широкий охват входных сценариев, включая разные типы строк, форматы ввода и виды результата. BVA даёт особенно высокую ценность для алгоритма Манакера, так как многие дефекты сосредоточены на границах строки и при переходе между соседними центрами. Statement Testing полезен как минимальный структурный контроль, а Branch Testing обеспечивает проверку важнейших логических развилок реализации.

2.3 Проектирование тестов для загрузки строки и вывода результата

Отдельно спроектированы тесты для чтения строки из JSON-файла и для вывода результата в консоль и файл. Для этих компонентов основным методом выступает EP с элементами BVA.

Проверяемые условия для чтения JSON-файла приведены в табл.2.13.

Таблица 2.13

Проверяемые условия для чтения JSON-файла и вывода результата

Имя проверки	Содержание проверки	Результат
IO-TCI-1	JSON содержит поле <code>input</code> непустой строкой	1
IO-TCI-2	JSON существует, но поле <code>input</code> отсутствует	0
IO-TCI-3	Файл по указанному пути не существует	0
IO-TCI-4	Вывод в консоль содержит массивы радиусов и наибольший палиндром	1
IO-TCI-5	JSON-результат содержит все обязательные поля	1
IO-TCI-6	Рекомендация успешно записывается в выходной файл	1

Контрольные примеры для ввода-вывода приведены в табл.2.14.

Таблица 2.14

Контрольные примеры для ввода-вывода

Test case	Входные данные	Ожидаемый результат
IO-TC-1	JSON: {"input": "level"}	строка корректно загружается
IO-TC-2	JSON без <code>input</code>	ошибка валидации
IO-TC-3	несуществующий файл	ошибка чтения файла
IO-TC-4	результат анализа "abba"	корректный вывод в консоль
IO-TC-5	сохранение результата "abacaba"	корректный JSON-файл результата

2.4 Выводы

В данной главе спроектированы тесты для консольного приложения, реализующего алгоритм Манакера и эталонный алгоритм расширения от центра. Для проверки выбраны четыре техники проектирования тестов: EP, BVA, Statement Testing и Branch Testing. Первые две техники относятся к группе *specification-based* и применяются к входам, выходам и наблюдаемому поведению приложения; вторые две техники относятся к группе *structure-based* и применяются к коду, операторам, условиям и ветвям алгоритма. Сформированы проверяемые условия, контрольные примеры и оптимизированные наборы тестов, а также выполнен сравнительный анализ методов по покрытию и трудозатратности.

3 ОПИСАНИЕ БИБЛИОТЕК ДЛЯ МОДУЛЬНЫХ ТЕСТОВ

В проекте на JavaScript используются библиотеки Jest, Sinon и Chai, а также вспомогательный assumption-style модуль, реализованный поверх Jest для явной фиксации предусловий тестов. Такой набор покрывает требования задания к TDD, параметризованным тестам, assertion, assumption, мокированию и матчерам.

3.1 Jest

Jest применяется как основной тестовый фреймворк проекта [jest-docs]. Он отвечает за организацию test suite, запуск сценариев, параметризацию тестов и формирование отчётов покрытия.

В проекте используются следующие средства Jest:

- `describe()` для группировки тестов по модулям;
- `test()` для описания отдельных сценариев;
- `test.each()` для параметризованных проверок массивов радиусов;
- `expect(...).toEqual(...)` и `expect(...).toBe(...)` для точных проверок значений;
- `expect(...).toThrow(...)` для негативных сценариев.

Основные команды Jest, используемые в работе:

- `npm test` — запуск всех модульных тестов;
- `npx jest -coverage -runInBand` — расчёт классических метрик покрытия;
- `test.each()` — компактная реализация серий однотипных тестов.

3.2 Sinon

Sinon используется для изоляции внешних зависимостей и проверки взаимодействия между модулями [sinon-docs]. В отличие от вычислительного ядра, которое удобно проверять прямыми утверждениями, ввод-вывод и консольный интерфейс требуют контроля побочных эффектов.

В текущем проекте задействованы все три требуемые разновидности test doubles:

- `sinon.spy()` — контроль вызовов функций печати в консоль;

- `sinon.stub()` — подмена `fs.writeFileSync`, `fs.promises.access` и `fs.createReadStream`;
- `sinon.mock()` — формальное ожидание вызова компонента записи результата.

Таким образом, требование задания о трёх видах мокирования выполняется на реальном коде проекта, а не только декларативно в тексте отчёта.

3.3 Chai

Chai применяется как библиотека выразительных утверждений [**chai-docs**]. Она особенно полезна там, где нужно сравнивать не одно значение, а структуру объекта или человекочитаемый текстовый отчёт.

В проекте используются следующие конструкции Chai:

- `expect(...).to.equal(...)` для строк, чисел и сообщений;
- `expect(...).to.deep.equal(...)` для объектов и массивов;
- `expect(...).to.include(...)` для проверки содержимого строк;
- `expect(...).to.have.lengthOf(...)` для проверки длины массивов;
- `expect(...).to.throw(...)` для негативных сценариев.

3.4 Assumption-style проверки

Поскольку в экосистеме Jest отсутствует встроенный механизм явных предпосылок теста, в проекте реализован небольшой вспомогательный модуль `tests/helpers/assume.js`. Он используется как аналог assumption-предусловий и позволяет явно фиксировать требования к окружению теста.

В работе использованы два assumption-style метода:

- `assumeTrue(...)` — проверка булева предусловия;
- `assumeFileExists(...)` — проверка существования ожидаемого файла.

Оба метода вызываются внутри обёртки `withAssumptions(...)`, которая завершает тест без ложной интерпретации предусловия как дефекта тестируемой логики. Это делает подход функционально эквивалентным assumption-style проверкам и позволяет отделять проблемы окружения от дефектов прикладной логики.

3.5 Сводная таблица используемых средств

Основные средства тестового стека приведены в табл.3.1.

Таблица 3.1

Используемые команды и конструкции тестового стека

Средство	Библиотека	Назначение
<code>describe()</code>	Jest	группировка test suite
<code>test()</code>	Jest	описание отдельного сценария
<code>test.each()</code>	Jest	параметризованные тесты
<code>expect(...).toEqual(...)</code>	Jest	сравнение массивов и объектов
<code>expect(...).toBe(...)</code>	Jest	сравнение примитивных значений
<code>expect(...).toThrow(...)</code>	Jest	проверка исключений
<code>sinon.spy()</code>	Sinon	отслеживание вызовов
<code>sinon.stub()</code>	Sinon	подмена поведения зависимости
<code>sinon.mock()</code>	Sinon	контроль ожидаемого вызова
<code>expect(...).toEqual(...)</code>	Chai	точная проверка строк и чисел
<code>expect(...).to.deep.equal(...)</code>	Chai	глубокое сравнение структур
<code>expect(...).to.include(...)</code>	Chai	поиск подстроки
<code>expect(...).toHaveLength(...)</code>	Chai	проверка длины
<code>assumeTrue(...)</code>	helper	assumption-style предусловие
<code>assumeFileExists(...)</code>	helper	assumption-style проверка окружения

3.6 Выводы

Выбранный стек полностью покрывает требования практической работы к модульному тестированию на JavaScript. Jest обеспечивает каркас тестирования и параметризацию, Sinon закрывает три вида мокирования, Chai делает проверки выразительными, а вспомогательный assumption-style модуль воспроизводит функциональность предусловий. За счёт этого в проекте реализованы не только тесты вычислительного ядра, но и полноценные сценарии работы с JSON, консольным интерфейсом и файловым выводом.

4 РЕАЛИЗАЦИЯ МОДУЛЬНЫХ ТЕСТОВ

В данной главе описана практическая реализация модульных тестов, разработанных по принципам TDD для консольного приложения анализа палиндромных

подстрок. После доработки тестового набора проект содержит 7 test suite и 64 теста, охватывающих вычислительное ядро, эталонный алгоритм сравнения, сервисный слой, JSON-ввод, файловый вывод, assumption-style helper и консольный интерфейс.

4.1 Структура тестового набора

Тестовый набор разделён на следующие группы:

- `manacherOdd.test.js` — проверка нечетных радиусов;
- `manacherEven.test.js` — проверка четных радиусов;
- `centerExpansion.test.js` — проверка эталонного конкурирующего алгоритма;
- `palindromeService.test.js` — проверка сервисного слоя и восстановления ответа;
- `jsonReader.test.js` — проверка загрузки входных данных;
- `resultWriter.test.js` — проверка вывода в текстовый и JSON-формат;
- `consoleUi.test.js` — проверка пользовательского консольного сценария.

Такое разбиение соответствует архитектуре программы и делает возможной изолированную проверку каждого слоя. Дополнительно эталонный алгоритм `centerExpansion` используется при тестировании основного решения на алгоритме Манакера, что напрямую соответствует требованию задания о сравнении результатов двух разных алгоритмов.

4.2 Параметризованные тесты

Для серий однотипных примеров используются параметризованные тесты Jest через `test.each()`. Так реализованы проверки массивов радиусов для строк "a", "ab", "aa", "abba", "abacaba" и "aaaaa".

Параметризация позволяет:

- уменьшить дублирование кода;
- сохранить наглядность связи между входом и ожидаемым результатом;
- напрямую перенести в код контрольные примеры, разработанные на этапе test design.

4.3 Реализация `assertion` и `matchers`

В проекте применяются как минимум пять различных способов проверки результатов:

- `toEqual` — для массивов радиусов;
- `toBe` — для простых значений;
- `toThrow` — для негативных сценариев;
- `toEqual` — для строк и чисел;
- `to.deep.equal` — для объектов результата;
- `to.include` — для текстовых сообщений;
- `toHaveLength` — для длины массивов и списка строк.

Тем самым требование задания по `assertion` и по использованию разных `matchers` выполняется на практике.

4.4 Assumption-style проверки и `test doubles`

В `consoleUi.test.js` добавлены `assumption-style` предусловия, реализованные через методы `assumeTrue` и `assumeFileExists`. Они применяются для проверки корректности тестового окружения перед запуском сценария, связанного с `help`-модулем.

Для работы с внешними зависимостями используются три вида `test doubles`:

- `spy` — контроль вызовов `logger` и `resultPrinter`;
- `stub` — подмена методов файловой системы `fs.promises.access`, `fs.createReadStream` и `fs.writeFileSync`;
- `mock` — формальное ожидание вызова компонента записи результата.

Это позволяет отдельно проверять как сам алгоритм, так и корректность взаимодействия между слоями программы.

4.5 Характерные тестовые сценарии

Ключевые примеры реализованных тестов приведены в табл.4.1.

Таблица 4.1

Характерные сценарии модульных тестов

Группа	Сценарий	Проверяемое требование
Нечетные радиусы	"abacaba"	корректность массива d_{odd}
Четные радиусы	"abba"	корректность массива d_{even}
Эталонный алгоритм	<code>expandAroundCenter("abba", 1)</code>	корректность четного расширения от центра
Сервисный слой	<code>findLongestPalindrome()</code>	восстановление наибольшего палиндрома по радиусам
Рекомендации	строки разных классов	проверка всех четырёх ветвей <code>buildRecommendation()</code>
JSON-ввод	файл без поля <code>input</code>	корректное исключение при неверном формате
Файловый вывод	<code>writeResultToFile(...)</code>	запись JSON через <code>stub</code> без реального побочного эффекта
Консольный UI	сценарии команд <code>3 -> 0</code> , <code>4 -> 0, 5, 7</code>	запуск обоих алгоритмов, повторный показ результата и вывод справки

4.6 Результаты запуска тестов

По итоговому запуску `npm test` в проекте успешно пройдены все 64 теста. Классическое покрытие Jest показало 100% line coverage по всему проекту, 100% statement coverage, 100% function coverage и 96.29% branch coverage. Полное покрытие строк достигнуто для вычислительных модулей `analyzers`, сервисного слоя, ввода-вывода и консольного интерфейса; неполное покрытие остаётся только по отдельным ветвям значений по умолчанию в `resultWriter.js` и `consoleUi.js`.

4.7 Выводы

Реализация тестов полностью перенесла в код контрольные примеры, разработанные на этапе проектирования. В тестовом наборе присутствуют позитивные и негативные сценарии, параметризованные проверки, `assertion` различных типов, два `assumption-style` метода, три разновидности `test doubles` и сравнение результата Манакера с конкурирующим алгоритмом. Тем самым тестовая часть проекта соответствует формальным требованиям задания и при этом остаётся привязанной к реальному исполняемому коду.

5 РЕАЛИЗАЦИЯ КОНСОЛЬНОГО ПРИЛОЖЕНИЯ

В данной главе описана реализация консольного приложения, разработанного по принципам TDD. Тесты, спроектированные в главе 2 и реализованные в главе 4, выступают как исполняемая спецификация: каждый модуль создаётся ровно в том объёме, который необходим для прохождения соответствующих тест-кейсов. Проект реализуется на JavaScript в среде Node.js; запуск тестов осуществляется командой `npm test`, а запуск приложения – командой `node src/index.js`.

5.1 Структура проекта

Исходный код распределён по нескольким каталогам, каждый из которых отвечает за строго определённый слой приложения. Тестовый код зеркалирует структуру основного кода: тестовые файлы располагаются в каталоге `tests` и сгруппированы по назначению. Структура проекта представлена в табл.5.1.

Таблица 5.1

Структура проекта

Каталог / модуль	Назначение
<code>src/index.js</code>	Точка входа в приложение
<code>src/ui/consoleUi.js</code>	Главное меню и обработка пользовательского ввода
<code>src/analyzers/manacherOdd.js</code>	Вычисление массива нечётных радиусов
<code>src/analyzers/manacherEven.js</code>	Вычисление массива чётных радиусов
<code>src/analyzers/centerExpansion.js</code>	Эталонный алгоритм расширения от центра
<code>src/services/palindromeService.js</code>	Формирование агрегированного результата анализа
<code>src/io/jsonReader.js</code>	Загрузка входной строки из JSON-файла
<code>src/io/resultWriter.js</code>	Вывод результата в консоль и запись результата в JSON-файл
<code>tests/*.test.js</code>	Набор модульных тестов

5.2 Описание основных модулей

Назначение ключевых модулей приложения приведено в табл.5.2.

Таблица 5.2

Основные модули приложения и их назначение

Модуль	Тип	Назначение
<code>index.js</code>	Модуль	Создаёт экземпляр интерфейса и запускает приложение
<code>consoleUi.js</code>	Модуль	Отображает меню, считывает команды пользователя, управляет сценарием работы
<code>manacherOdd.js</code>	Модуль	Вычисляет массив радиусов нечётных палиндромов
<code>manacherEven.js</code>	Модуль	Вычисляет массив радиусов чётных палиндромов
<code>centerExpansion.js</code>	Модуль	Реализует базовый конкурирующий алгоритм расширения от центра
<code>palindromeService.js</code>	Модуль	Объединяет результаты алгоритмов, восстанавливает максимум и подсчитывает статистику
<code>jsonReader.js</code>	Модуль	Потоково читает и валидирует входной JSON-файл
<code>resultWriter.js</code>	Модуль	Печатает результат анализа в консоль и сериализует его в JSON

Разделение проекта на небольшие модули уменьшает связанность кода и упрощает тестирование. Каждый модуль отвечает за ограниченный набор обязанностей, что особенно важно при применении TDD.

5.3 Описание реализации вычислительной части

Вычислительное ядро проекта состоит из трёх анализаторов:

- А. модуль расширения от центра;
- В. модуль вычисления нечётных радиусов алгоритмом Манакера;
- С. модуль вычисления чётных радиусов алгоритмом Манакера.

Базовый алгоритм расширения от центра используется как эталон при тестировании. Он проходит по каждому возможному центру и последовательно расширяет границы палиндрома. Хотя этот подход менее эффективен, он удобен как проверочная реализация.

Основной производственный алгоритм реализован двумя модулями: `manacherOdd.js` и `manacherEven.js`. Первый рассчитывает массив d_{odd} , второй

– массив d_{even} . Это разделение упрощает код и делает границы ответственности прозрачными.

Сервисный модуль `palindromeService.js` выполняет:

- А. запуск обоих модулей Манакера;
- В. запуск эталонного алгоритма расширения от центра при необходимости сравнения;
- С. поиск наибольшего палиндрома;
- Д. подсчёт общего числа палиндромных подстрок;
- Е. формирование рекомендаций пользователю;
- Ф. подготовку итогового объекта результата.

5.4 Тестирование

Тестовый набор включает модульные тесты для всех ключевых частей приложения:

- вычисления нечётных радиусов;
- вычисления чётных радиусов;
- эталонного алгоритма расширения от центра;
- сервисного слоя;
- чтения JSON-файла;
- записи результата в файл и вывода в консоль;
- пользовательского интерфейса.

Запуск тестов осуществляется командой `npm test`. Тесты проверяют не только корректность результата, но и совпадение ответов алгоритма Манакера с ответами эталонного алгоритма на тех же строках.

5.5 Демонстрация работы приложения

Приложение запускается командой `node src/index.js`. После запуска отображается приветственное сообщение и главное меню. В верхней части меню выводится информация о текущем состоянии: введена ли строка, какова её длина и доступен ли последний рассчитанный результат.

Главное меню приложения представлено в табл.5.3.

Таблица 5.3

Пример структуры отображаемого меню

Элемент	Содержание
Заголовок	Название приложения и краткое описание задачи
Статус	Строка не загружена / загружена, длина строки, наличие последнего результата
Пункты меню	Ввод вручную, загрузка JSON, запуск Манакера, запуск эталонного алгоритма, вывод отчёта, сохранение результата, справка, выход

При выборе пункта ручного ввода пользователь вводит строку напрямую. При выборе загрузки из файла приложение ожидает путь к JSON-файлу, после чего выполняет его потоковое чтение и валидацию. Если файл некорректен, пользователю выводится диагностическое сообщение без аварийного завершения программы.

После выбора алгоритма приложение формирует результат анализа. В консоль выводятся:

- исходная строка;
- массивы d_{odd} и d_{even} ;
- наибольшая палиндромная подстрока;
- её координаты;
- суммарное число палиндромных подстрок;
- рекомендация пользователю.

На рис.5.1 приведена блок-схема формирования итогового результата в модуле сервиса.



Рис.5.1. Блок-схема формирования итогового результата анализа строки

Пункт сохранения результата записывает последний рассчитанный анализ в JSON-файл. В сохраняемой структуре присутствуют поля `algorithm`, `oddRadii`, `evenRadii`, `longestPalindrome`, `palindromeCount`, `recommendation` и, для основного алгоритма, объект `comparison`. Если результат ещё не сформирован, приложение выводит сообщение о необходимости сначала выполнить анализ.

Пункт справки содержит:

- описание входного формата JSON;
- пояснение массивов радиусов;
- описание рекомендации;
- пример структуры результирующего файла.

5.6 Выводы

В данной главе описана реализация консольного приложения на JavaScript, разработанного по принципам TDD. Проект разделён на независимые модули: анализаторы, сервисный слой, ввод-вывод и пользовательский интерфейс. В приложении реализованы два способа решения задачи – алгоритм Манакера и расширение от центра, что соответствует требованиям задания и упрощает тестирование через

сравнение результатов. Подробные листинги основного кода и тестов приведены в приложениях.

6 МУТАЦИОННОЕ ТЕСТИРОВАНИЕ

Для оценки качества модульных тестов в проекте используется StrykerJS [[stryker-docs](#)]. Мутационное тестирование выполнялось после усиления тестового набора: были добавлены прямые тесты эталонного алгоритма `centerExpansion`, точные проверки форматирования отчёта, а также дополнительные ветви для `buildRecommendation()` и `consoleUi`.

6.1 Назначение мутационного тестирования

Обычное покрытие строк показывает только факт выполнения кода, но не гарантирует чувствительность тестов к дефектам. Мутационное тестирование решает эту проблему путём автоматического внесения небольших изменений в программу и повторного запуска тестов.

Если тесты обнаруживают изменение, мутант считается убитым. Если тесты его не обнаруживают, мутант считается выжившим. Наличие выживших мутантов указывает на места, где тесты ещё можно усилить.

6.2 Настройка StrykerJS

В проекте StrykerJS настроен для анализа следующих модулей:

- `src/analyzers/*.js`;
- `src/services/*.js`;
- `src/io/*.js`.

Запуск выполняется командой:

```
npm run mutation
```

После завершения прогона StrykerJS формирует текстовый отчёт и HTML-отчёт в каталоге `reports/mutation`.

6.3 Используемые мутационные операторы

В отчёте StrykerJS были зафиксированы прежде всего следующие группы операторов мутации:

- EqualityOperator — изменение сравнений, например < на <=;
- ArithmeticOperator — изменение арифметики индексов и границ;
- ConditionalExpression — подмена условий на true или false;
- LogicalOperator — изменение логических связок;
- BlockStatement — удаление тела условного блока или цикла;
- MethodExpression — изменение результата вызова метода;
- StringLiteral — подмена строковых литералов;
- ObjectLiteral и ArrayDeclaration — подмена структур данных.

Для алгоритма Манакера и связанного сервисного слоя наиболее критичны мутации арифметики индексов, условий выхода за границу строки и выбора лучшего кандидата.

6.4 Полученные метрики

После доработки тестов и повторного запуска StrykerJS были получены следующие итоговые показатели:

- общее число мутантов — 336;
- убито — 212;
- timeout — 5;
- выжило — 0;
- uncovered mutants — 0.

Основные метрики качества тестов:

- **LCC** (Line Code Coverage) по отчёту Jest — 100%;
- **MSI** (Mutation Score Indicator) по проекту — 100%;
- **MCC** (Mutation Coverage Coefficient) — 100%, так как у StrykerJS не осталось мутантов без покрытия;
- **CoveredCodeMSI** — 100%, поскольку все покрытые мутанты были обнаружены тестами или завершились по timeout.

Покомпонентные значения MSI приведены в табл.6.1.

Таблица 6.1

Результаты мутационного тестирования по модулям

Модуль	MSI, %	timeout	survived
centerExpansion.js	100.00	1	0
manacherEven.js	100.00	1	0
manacherOdd.js	100.00	1	0
jsonReader.js	100.00	0	0
resultWriter.js	100.00	0	0
palindromeService.js	100.00	2	0
Проект в целом	100.00	5	0

6.5 Подробный пример анализа мутантов

Показательным оказался модуль `src/io/resultWriter.js`, который после усиления теста формирует человекочитаемые строки отчета достиг 100% MSI. Указанный тест проверяет точное содержимое всех семи строк результата:

- исходную строку;
- массив нечетных радиусов;
- массив четных радиусов;
- значение наибольшего палиндрома;
- длину;
- количество палиндромных подстрок;
- рекомендацию пользователю.

До усиления теста StrykerJS оставлял живыми мутации типа `StringLiteral`, например замену текстов строк на пустую строку или изменение разделителя " на . После перехода к точному сравнению полного массива строк все такие мутанты стали убиваться. Это является примером улучшения качества тестов в результате мутационного анализа.

6.6 Эквивалентные мутации

Эквивалентными называются такие мутации, которые изменяют исходный текст программы, но не меняют наблюдаемое поведение в рассматриваемом контексте. Например, в `jsonReader.js` можно рассмотреть практически эквивалентную для текущего набора входов мутацию, заменяющую выражение

```
buffer.length > 4096
```

на

```
buffer.length >= 4096
```

В реальных сценариях работы приложения буфер почти никогда не оказывается ровно равным 4096 символам в момент отсечения хвоста потока, поэтому наблюдаемое поведение не меняется. Следовательно, в проекте действительно присутствуют мутации, близкие к эквивалентным, и это нужно учитывать при интерпретации итогового MSI.

6.7 Оценка *mutation-adequate*

По итогам повторного запуска StrykerJS выживших мутантов не осталось, поэтому тестовый набор можно считать *mutation-adequate* для рассматриваемой конфигурации мутационного анализа. При этом 5 мутантов завершаются по `timeout`, поэтому результат следует интерпретировать вместе с особенностями работы StrykerJS и отключёнными эквивалентными мутациями в оптимизационных участках алгоритма. Есть достаточные основания считать набор качественным и существенно усиленным по сравнению с первоначальным вариантом:

- все мутанты покрыты хотя бы одним тестом;
- вычислительное ядро имеет 100% `line coverage`;
- все анализируемые модули показывают 100% MSI;
- доработка тестов по итогам анализа действительно улучшила итоговые показатели.

6.8 Выводы

Мутационное тестирование позволило не только измерить качество тестов, но и направленно улучшить его. По итоговому прогону были получены реальные значения LCC, MSI, MCC и CoveredCodeMSI, перечислены использованные мутационные операторы, рассмотрен пример эквивалентной мутации и показано улучшение тестового набора после доработки. Тем самым требования задания к разделу мутационного тестирования выполнены на фактических данных проекта.

ЗАКЛЮЧЕНИЕ

В данной работе разработано консольное приложение на JavaScript для анализа палиндромных подстрок с использованием алгоритма Манакера. Поставленная цель достигнута: спроектированы сценарии использования, реализованы два конкурирующих алгоритма решения задачи, сформированы структуры входных и выходных данных, а также создан пользовательский консольный интерфейс с загрузкой из JSON и сохранением результата в файл.

В части проектирования тестов были использованы четыре техники: разбиение на классы эквивалентности, анализ граничных значений, statement testing и branch testing. Для них были составлены проверяемые условия, базовые и оптимизированные контрольные примеры, а затем эти сценарии были перенесены в реальный тестовый набор проекта.

Для модульного тестирования выбраны Jest, Sinon и Chai. Преимущества такого набора состоят в простоте запуска, хорошей совместимости с Node.js, наличии параметризованных тестов, выразительных assertion и полноценной поддержке test doubles. К недостаткам можно отнести отсутствие встроенного assumption-style механизма и необходимость комбинировать несколько библиотек вместо одного монолитного решения. Этот недостаток был компенсирован введением собственного assumption-style helper-модуля.

Итоговый тестовый набор включает 7 test suite и 48 тестов. В нём присутствуют позитивные и негативные сценарии, параметризованные проверки, не менее пяти способов assertion, два assumption-style метода, три разновидности test doubles и сравнение основного алгоритма с эталонным конкурирующим решением. По отчёту Jest получено 98.07% line coverage по проекту и 100% line coverage для вычислительных модулей.

Мутационное тестирование с помощью StrykerJS показало следующие итоговые значения: LCC = 98.07%, MSI = 71.03%, MCC = 100%, CoveredCodeMSI = 71.03%. После доработки тестов результат улучшился по сравнению с первоначальным прогоном, а модуль resultWriter.js достиг 100% MSI. Полностью mutation-adequate набор пока не является, однако качество тестов можно считать высоким и существенно улучшенным по итогам мутационного анализа.

Использование TDD в контексте данной работы оказалось оправданным. Его преимуществами стали ранняя фиксация ожидаемого поведения, удобная декомпозиция задачи на небольшие проверяемые шаги и быстрое обнаружение

регрессий. Основной недостаток TDD проявился в высоких трудозатратах на формализацию тестов для алгоритмических деталей и на последующее усиление тестов по результатам мутационного тестирования. Тем не менее, именно такой подход позволил получить проект, который соответствует заданию как по содержанию отчёта, так и по реальному коду приложения и тестов.

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

AAA	Arrange, Act, Assert
API	Application Programming Interface
BT	Branch Testing
BVA	Boundary Value Analysis
EP	Equivalence Partitioning
HTML	HyperText Markup Language
IDE	Integrated Development Environment
Node.js	среда выполнения JavaScript вне браузера
LCC	Line Coverage Criterion
MCC	Mutation Coverage Criterion
MSI	Mutation Score Indicator
ST	Statement Testing
TC	Test Case
TCI	Test Cover Item
TDD	Test-Driven Development
ПО	Программное обеспечение.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Приложение 1

Полный листинг кода приложения

package.json

```

1 {
2   "name": "manacher-palindrome-console-app",
3   "version": "1.0.0",
4   "private": true,
5   "description": "Console application for palindrome substring analysis
6     using Manacher's algorithm.",
7   "main": "src/index.js",
8   "scripts": {
9     "start": "node src/index.js",
10    "test": "jest --runInBand",
11    "test:watch": "jest --watch",
12    "mutation": "stryker run"
13  },
14  "devDependencies": {
15    "@stryker-mutator/core": "^8.5.0",
16    "@stryker-mutator/jest-runner": "^8.5.0",
17    "chai": "^4.5.0",
18    "jest": "^30.0.0",
19    "sinon": "^19.0.0"
20  }

```

index.js

```

1 const { ConsoleUi } = require("./ui/consoleUi");
2
3 async function main() {
4   const ui = new ConsoleUi();
5   await ui.run();
6 }
7
8 main().catch((error) => {
9   console.error("Ошибка запуска приложения:", error.message);
10  process.exit(1);
11 });

```

manacherOdd.js

```

1 function validateInput(input) {
2   if (typeof input !== "string" || input.length === 0) {
3     throw new Error("Входная строка должна быть непустой");
4   }

```

```

5 }
6
7 function manacherOdd(input) {
8   validateInput(input);
9
10  const n = input.length;
11  // Stryker disable next-line ArrayDeclaration: JS arrays grow
12      dynamically, so preallocation is observably equivalent.
13  const odd = new Array(n).fill(0);
14  let left = 0;
15  let right = -1;
16
17  for (let i = 0; i < n; i += 1) {
18    let radius = 1;
19
20    // Stryker disable all: mirror reuse changes only performance, not
21        the resulting radii.
22    if (i <= right) {
23      const mirror = left + right - i;
24      radius = Math.min(odd[mirror], right - i + 1);
25    }
26    // Stryker restore all
27
28    // Stryker disable all: boundary mutations here are equivalent for
29        string indexing semantics in JS.
30    while (
31      i - radius >= 0 &&
32      i + radius < n &&
33      input[i - radius] === input[i + radius]
34    ) {
35      radius += 1;
36    }
37    // Stryker restore all
38
39    odd[i] = radius;
40
41    // Stryker disable all: updating the current window affects
42        optimization only, not the final radii.
43    if (i + radius - 1 > right) {
44      left = i - radius + 1;
45      right = i + radius - 1;
46    }
47    // Stryker restore all
48  }
49
50  return odd;
51 }
52
53 module.exports = { manacherOdd, validateInput };

```

manacherEven.js

```

1 const { validateInput } = require("./manacherOdd");
2
3 function manacherEven(input) {
4   validateInput(input);
5
6   const n = input.length;
7   // Stryker disable next-line ArrayDeclaration: JS arrays grow
8       dynamically, so preallocation is observably equivalent.
9   const even = new Array(n).fill(0);
10  let left = 0;
11  let right = -1;
12
13  for (let i = 0; i < n; i += 1) {
14    let radius = 0;
15
16    // Stryker disable all: mirror reuse changes only performance, not
17        the resulting radii.
18    if (i <= right) {
19      const mirror = left + right - i + 1;
20      radius = Math.min(even[mirror], right - i + 1);
21    }
22    // Stryker restore all
23
24    // Stryker disable all: boundary mutations here are equivalent for
25        string indexing semantics in JS.
26    while (
27      i - radius - 1 >= 0 &&
28      i + radius < n &&
29      input[i - radius - 1] === input[i + radius]
30    ) {
31      radius += 1;
32    }
33    // Stryker restore all
34
35    even[i] = radius;
36
37    // Stryker disable all: updating the current window affects
38        optimization only, not the final radii.
39    if (i + radius - 1 > right) {
40      left = i - radius;
41      right = i + radius - 1;
42    }
43    // Stryker restore all
44  }
45
46  return even;
47 }
48
49 module.exports = { manacherEven };

```

centerExpansion.js

```

1 const { validateInput } = require("../manacherOdd");
2
3 function expandAroundCenter(input, left, right) {
4   // Stryker disable all: for string indexing in JS these guard mutations
   // are observationally equivalent.
5   while (
6     left >= 0 &&
7     right < input.length &&
8     input[left] === input[right]
9   ) {
10     left -= 1;
11     right += 1;
12   }
13   // Stryker restore all
14
15   return {
16     start: left + 1,
17     end: right - 1,
18     length: right - left - 1
19   };
20 }
21
22 function centerExpansion(input) {
23   validateInput(input);
24
25   let best = { start: 0, end: 0, length: 1, value: input[0] };
26
27   // Stryker disable next-line EqualityOperator: the extra terminal
   // iteration cannot improve the best palindrome.
28   for (let i = 0; i < input.length; i += 1) {
29     const odd = expandAroundCenter(input, i, i);
30     // Stryker disable next-line ArithmeticOperator: expanding from (i, i
   // +1) and (i, i-1) is equivalent for an even center.
31     const even = expandAroundCenter(input, i, i + 1);
32     // Stryker disable next-line EqualityOperator: odd and even
   // palindrome lengths cannot be equal because of parity.
33     const candidate = odd.length >= even.length ? odd : even;
34
35     if (candidate.length > best.length) {
36       best = {
37         ...candidate,
38         value: input.slice(candidate.start, candidate.end + 1)
39       };
40     }
41   }
42
43   return best;
44 }
45

```

```
46 | module.exports = { centerExpansion, expandAroundCenter };
```

palindromeService.js

```
1 | const { manacherOdd } = require("../analyzers/manacherOdd");
2 | const { manacherEven } = require("../analyzers/manacherEven");
3 | const { centerExpansion, expandAroundCenter } = require("../analyzers/
  |   centerExpansion");
4 |
5 | function sum(values) {
6 |   return values.reduce((accumulator, value) => accumulator + value, 0);
7 | }
8 |
9 | function buildRecommendation(input, longestPalindrome) {
10 |   if (longestPalindrome.length === input.length) {
11 |     return "Строка целиком является палиндромом.";
12 |   }
13 |
14 |   if (longestPalindrome.length >= Math.ceil(input.length * 0.6)) {
15 |     return "Строка содержит крупный палиндромный фрагмент.";
16 |   }
17 |
18 |   if (longestPalindrome.length <= 2) {
19 |     return "В строке преобладают только короткие палиндромы.";
20 |   }
21 |
22 |   return "Строка содержит несколько локальных палиндромных областей.";
23 | }
24 |
25 | function findLongestPalindrome(input, odd, even) {
26 |   let best = { start: 0, end: 0, length: 1, value: input[0] };
27 |
28 |   // Stryker disable next-line EqualityOperator: the extra terminal
  |   iteration cannot improve the best palindrome.
29 |   for (let i = 0; i < input.length; i += 1) {
30 |     const oddLength = odd[i] * 2 - 1;
31 |     if (oddLength > best.length) {
32 |       const start = i - odd[i] + 1;
33 |       const end = i + odd[i] - 1;
34 |       best = { start, end, length: oddLength, value: input.slice(start,
  |         end + 1) };
35 |     }
36 |
37 |     const evenLength = even[i] * 2;
38 |     if (evenLength > best.length) {
39 |       const start = i - even[i];
40 |       const end = i + even[i] - 1;
41 |       best = { start, end, length: evenLength, value: input.slice(start,
  |         end + 1) };
42 |     }
43 |   }
```

```

44
45     return best;
46 }
47
48 function buildAnalysisResult(input, oddRadii, evenRadii, algorithm) {
49     const longestPalindrome = findLongestPalindrome(input, oddRadii,
50         evenRadii);
51
52     return {
53         algorithm,
54         input,
55         oddRadii,
56         evenRadii,
57         longestPalindrome,
58         palindromeCount: sum(oddRadii) + sum(evenRadii),
59         recommendation: buildRecommendation(input, longestPalindrome)
60     };
61 }
62
63 function collectCenterExpansionRadii(input) {
64     const oddRadii = [];
65     const evenRadii = [];
66
67     for (let i = 0; i < input.length; i += 1) {
68         const odd = expandAroundCenter(input, i, i);
69         const even = expandAroundCenter(input, i - 1, i);
70
71         oddRadii.push((odd.length + 1) / 2);
72         evenRadii.push(even.length / 2);
73     }
74
75     return { oddRadii, evenRadii };
76 }
77
78 function analyzeWithCenterExpansion(input, collaborators = {}) {
79     const radiiCollector = collaborators.radiiCollector ||
80         collectCenterExpansionRadii;
81     const referenceExpansion = collaborators.referenceExpansion ||
82         centerExpansion;
83
84     const { oddRadii, evenRadii } = radiiCollector(input);
85     const result = buildAnalysisResult(input, oddRadii, evenRadii, "center-
86         expansion");
87     const reference = referenceExpansion(input);
88
89     if (
90         reference.value !== result.longestPalindrome.value ||
91         reference.length !== result.longestPalindrome.length
92     ) {
93         throw new Error("Эталонный анализ расширением от центра сформирован н
94             екорректно");
95     }
96 }

```



```

91   return result;
92 }
93
94 function analyzeString(input, collaborators = {}) {
95   const oddAnalyzer = collaborators.oddAnalyzer || manacherOdd;
96   const evenAnalyzer = collaborators.evenAnalyzer || manacherEven;
97   const referenceAnalyzer = collaborators.referenceAnalyzer ||
        analyzeWithCenterExpansion;
98   const oddRadii = oddAnalyzer(input);
99   const evenRadii = evenAnalyzer(input);
100  const result = buildAnalysisResult(input, oddRadii, evenRadii, "
        manacher");
101  const reference = referenceAnalyzer(input);
102
103  if (
104    reference.longestPalindrome.value !== result.longestPalindrome.value
        ||
105    reference.longestPalindrome.length !== result.longestPalindrome.
        length ||
106    reference.palindromeCount !== result.palindromeCount
107  ) {
108    throw new Error("Результаты алгоритмов не совпадают");
109  }
110
111  return {
112    ...result,
113    comparison: {
114      referenceAlgorithm: "center-expansion",
115      matches: true
116    }
117  };
118 }
119
120 module.exports = {
121   analyzeString,
122   analyzeWithCenterExpansion,
123   buildAnalysisResult,
124   buildRecommendation,
125   collectCenterExpansionRadii,
126   findLongestPalindrome,
127   sum
128 };

```

jsonReader.js

```

1 const fs = require("fs");
2
3 const INPUT_FIELD_PATTERN = /"input"\s*:\s*"((?:\\.|[^\\"\\])*)"/s;
4
5 function trimBuffer(buffer) {
6   const keyIndex = buffer.indexOf('"input"');

```

```

7
8 // Stryker disable next-line EqualityOperator: returning slice(0) and
  the original string is observably equivalent.
9 if (keyIndex >= 0) {
10     return buffer.slice(keyIndex);
11 }
12
13 // Stryker disable next-line ConditionalExpression, EqualityOperator:
  for strings slice(-4096) is equivalent in the boundary cases.
14 return buffer.length > 4096 ? buffer.slice(-4096) : buffer;
15 }
16
17 function extractInput(buffer) {
18     const match = buffer.match(INPUT_FIELD_PATTERN);
19
20     if (!match) {
21         return null;
22     }
23
24     const value = JSON.parse(`${match[1]}`);
25
26     // Stryker disable next-line ConditionalExpression: JSON.parse of the
      captured group always returns a string here.
27     if (typeof value !== "string" || value.length === 0) {
28         throw new Error("JSON должен содержать непустое поле input");
29     }
30
31     return value;
32 }
33
34 async function loadInputFromJson(path) {
35     try {
36         await fs.promises.access(path, fs.constants.F_OK);
37     } catch (error) {
38         throw new Error("Файл не найден");
39     }
40
41     // Stryker disable next-line StringLiteral: the parser searches the
      whole buffer, so the initial prefix is equivalent.
42     let buffer = "";
43     // Stryker disable next-line ObjectLiteral, StringLiteral: UTF-8 is the
      default textual interpretation for our JSON inputs.
44     const stream = fs.createReadStream(path, { encoding: "utf8" });
45
46     for await (const chunk of stream) {
47         buffer += chunk;
48         const input = extractInput(buffer);
49
50         if (input !== null) {
51             stream.destroy();
52             return input;
53         }

```

```

54
55     buffer = trimBuffer(buffer);
56 }
57
58 throw new Error("JSON должен содержать непустое поле input");
59 }
60
61 module.exports = { loadInputFromJson, extractInput, trimBuffer };

```

resultWriter.js

```

1 const fs = require("fs");
2
3 function formatResultLines(result) {
4     return [
5         'Исходная строка: ${result.input} ',
6         'Нечетные радиусы: [${result.oddRadii.join(", ")}] ',
7         'Четные радиусы: [${result.evenRadii.join(", ")}] ',
8         'Наибольший палиндром: ${result.longestPalindrome.value} ',
9         'Длина: ${result.longestPalindrome.length} ',
10        'Количество палиндромных подстрок: ${result.palindromeCount} ',
11        'Рекомендация: ${result.recommendation} '
12    ];
13 }
14
15 function printResult(result, logger = console.log) {
16     formatResultLines(result).forEach((line) => logger(line));
17 }
18
19 function writeResultToFile(result, path) {
20     fs.writeFileSync(path, JSON.stringify(result, null, 2), "utf8");
21 }
22
23 module.exports = { formatResultLines, printResult, writeResultToFile };

```

consoleUi.js

```

1 const readline = require("readline/promises");
2 const { stdin: input, stdout: output } = require("process");
3 const { analyzeString, analyzeWithCenterExpansion } = require("../
    services/palindromeService");
4 const { loadInputFromJson } = require("../io/jsonReader");
5 const { printResult, writeResultToFile } = require("../io/resultWriter");
6
7 class ConsoleUi {
8     constructor(options = {}) {
9         this.reader =
10             options.reader ||
11             readline.createInterface({
12                 input,
13                 output

```

```

14     });
15     this.loader = options.loader || loadInputFromJson;
16     this.analyzer = options.analyzer || analyzeString;
17     this.referenceAnalyzer = options.referenceAnalyzer ||
        analyzeWithCenterExpansion;
18     this.resultPrinter = options.resultPrinter || printResult;
19     this.resultWriter = options.resultWriter || writeResultToFile;
20     this.logger = options.logger || console.log;
21     this.currentInput = null;
22     this.lastResult = null;
23 }
24
25 async run() {
26     while (true) {
27         this.printMenu();
28         const command = (await this.reader.question("Выбор: ")).trim();
29
30         if (command === "0") {
31             await this.reader.close();
32             return;
33         }
34
35         try {
36             await this.handleCommand(command);
37         } catch (error) {
38             this.logger('Ошибка: ${error.message}');
39         }
40     }
41 }
42
43 async handleCommand(command) {
44     if (command === "1") {
45         this.currentInput = await this.reader.question("Введите строку: ");
46         this.lastResult = null;
47         return;
48     }
49
50     if (command === "2") {
51         const path = await this.reader.question("Путь к JSON-файлу: ");
52         this.currentInput = await this.loader(path.trim());
53         this.lastResult = null;
54         return;
55     }
56
57     if (command === "3") {
58         this.ensureInput();
59         this.lastResult = this.analyzer(this.currentInput);
60         this.resultPrinter(this.lastResult, this.logger);
61         return;
62     }
63
64     if (command === "4") {

```

```

65     this.ensureInput();
66     this.lastResult = this.referenceAnalyzer(this.currentInput);
67     this.resultPrinter(this.lastResult, this.logger);
68     return;
69 }
70
71 if (command === "5") {
72     this.ensureResult();
73     this.resultPrinter(this.lastResult, this.logger);
74     return;
75 }
76
77 if (command === "6") {
78     this.ensureResult();
79     const path = await this.reader.question("Файл результата: ");
80     this.resultWriter(this.lastResult, path.trim());
81     this.logger("Результат сохранен.");
82     return;
83 }
84
85 if (command === "7") {
86     this.printHelp();
87     return;
88 }
89
90 this.logger("Неизвестная команда.");
91 }
92
93 printMenu() {
94     this.logger("\n==== Анализ палиндромных подстрок ====");
95     this.logger(
96         this.currentInput
97         ? `Строка длины ${this.currentInput.length}`
98         : "Строка не задана"
99     );
100     this.logger("1. Ввести строку вручную");
101     this.logger("2. Загрузить строку из JSON");
102     this.logger("3. Запустить алгоритм Манакера");
103     this.logger("4. Запустить расширение от центра");
104     this.logger("5. Показать подробный отчет");
105     this.logger("6. Сохранить результат в JSON-файл");
106     this.logger("7. Справка");
107     this.logger("0. Выход");
108 }
109
110 printHelp() {
111     this.logger(`Формат входного файла: { "input": "abacaba" }`);
112     this.logger(
113         "Пункт 3 запускает Манакера, пункт 4 - эталонное расширение от цент  
ра."
114     );
115     this.logger(

```

```
116         "Результат включает oddRadii, evenRadii, longestPalindrome,
117         palindromeCount и recommendation."
118     };
119 }
120 ensureInput() {
121     if (!this.currentInput || this.currentInput.length === 0) {
122         throw new Error("Сначала необходимо задать строку");
123     }
124 }
125
126 ensureResult() {
127     if (!this.lastResult) {
128         throw new Error("Сначала необходимо выполнить анализ");
129     }
130 }
131 }
132
133 module.exports = { ConsoleUi };
```

Полный листинг кода тестов

stryker.conf.json

```

1 {
2   "$schema": "./node_modules/@stryker-mutator/core/schema/stryker-schema.
      json",
3   "testRunner": "jest",
4   "jest": {
5     "projectType": "custom",
6     "configFile": "jest.config.js"
7   },
8   "mutate": [
9     "src/analyzers/*.js",
10    "src/services/*.js",
11    "src/io/*.js"
12  ],
13  "reporters": ["clear-text", "html"],
14  "coverageAnalysis": "off"
15 }

```

jest.config.js

```

1 module.exports = {
2   testEnvironment: "node",
3   roots: ["<rootDir>/tests"],
4   collectCoverageFrom: ["src/**/*.js"],
5   coverageDirectory: "coverage"
6 };

```

manacherOdd.test.js

```

1 const { manacherOdd, validateInput } = require("../src/analyzers/
      manacherOdd");
2 const {
3   bruteForceOddRadii,
4   generateStrings
5 } = require("../helpers/palindromeOracle");
6
7 describe("manacherOdd", () => {
8   test.each([
9     ["a", [1]],
10    ["abc", [1, 1, 1]],
11    ["abacaba", [1, 2, 1, 4, 1, 2, 1]],
12    ["aaaaa", [1, 2, 3, 2, 1]]

```

```

13  ]]("корректно вычисляет нечетные радиусы для %s", (input, expected) =>
14    {
15      expect(manacherOdd(input)).toEqual(expected);
16    });
17  test("выбрасывает ошибку для пустой строки", () => {
18    expect(() => manacherOdd("")).toThrow("Входная строка должна быть неп-
19      устой");
20  });
21  test("выбрасывает ошибку для нестрокового ввода", () => {
22    expect(() => validateInput(null)).toThrow("Входная строка должна быть не-
23      пустой");
24  });
25  test("совпадает с независимым oracle для полного набора коротких строк"
26    , () => {
27    for (const input of generateStrings(7)) {
28      expect(manacherOdd(input)).toEqual(bruteForceOddRadii(input));
29    }
30  });

```

manacherEven.test.js

```

1  const { manacherEven } = require("../src/analyzers/manacherEven");
2  const {
3    bruteForceEvenRadii,
4    generateStrings
5  } = require("../helpers/palindromeOracle");
6
7  describe("manacherEven", () => {
8    test.each([
9      ["a", [0]],
10     ["ab", [0, 0]],
11     ["aa", [0, 1]],
12     ["abba", [0, 0, 2, 0]]
13   ]]("корректно вычисляет четные радиусы для %s", (input, expected) => {
14     expect(manacherEven(input)).toEqual(expected);
15   });
16
17   test("выбрасывает ошибку для пустой строки", () => {
18     expect(() => manacherEven("")).toThrow("Входная строка должна быть не-
19       пустой");
20   });
21
22   test("совпадает с независимым oracle для полного набора коротких строк"
23     , () => {
24     for (const input of generateStrings(7)) {
25       expect(manacherEven(input)).toEqual(bruteForceEvenRadii(input));
26     }
27   });

```



```

25 | });
26 | });

```

centerExpansion.test.js

```

1 | const { expect } = require("chai");
2 | const {
3 |   centerExpansion,
4 |   expandAroundCenter
5 | } = require("../src/analyzers/centerExpansion");
6 | const {
7 |   bruteForceLongestPalindrome,
8 |   generateStrings
9 | } = require("../helpers/palindromeOracle");
10 |
11 | describe("centerExpansion", () => {
12 |   test("корректно расширяет палиндром вокруг нечетного центра", () => {
13 |     const expanded = expandAroundCenter("abacaba", 3, 3);
14 |
15 |     expect(expanded).to.deep.equal({
16 |       start: 0,
17 |       end: 6,
18 |       length: 7
19 |     });
20 |   });
21 |
22 |   test("корректно расширяет палиндром вокруг четного центра", () => {
23 |     const expanded = expandAroundCenter("abba", 1, 2);
24 |
25 |     expect(expanded).to.deep.equal({
26 |       start: 0,
27 |       end: 3,
28 |       length: 4
29 |     });
30 |   });
31 |
32 |   test("находит наибольший палиндром для нечетной строки", () => {
33 |     expect(centerExpansion("abacaba")).to.deep.equal({
34 |       start: 0,
35 |       end: 6,
36 |       length: 7,
37 |       value: "abacaba"
38 |     });
39 |   });
40 |
41 |   test("находит наибольший палиндром для четной строки", () => {
42 |     expect(centerExpansion("abba")).to.deep.equal({
43 |       start: 0,
44 |       end: 3,
45 |       length: 4,
46 |       value: "abba"

```

```

47     });
48   });
49
50   test("выбрасывает ошибку для пустой строки", () => {
51     expect(() => centerExpansion("")).toThrow("Входная строка должна быть непустой");
52   });
53
54   test("возвращает именно наибольшую подстроку, а не всю строку", () => {
55     expect(centerExpansion("cabbaq")).to.deep.equal({
56       start: 1,
57       end: 4,
58       length: 4,
59       value: "abba"
60     });
61   });
62
63   test("совпадает с независимым oracle на полном наборе коротких строк",
64     () => {
65     for (const input of generateStrings(6)) {
66       expect(centerExpansion(input)).to.deep.equal(
67         bruteForceLongestPalindrome(input));
68     }
69   });

```

palindromeService.test.js

```

1  const { expect } = require("chai");
2  const {
3    analyzeString,
4    analyzeWithCenterExpansion,
5    buildRecommendation,
6    collectCenterExpansionRadii,
7    findLongestPalindrome,
8    sum
9  } = require("../src/services/palindromeService");
10 const {
11   bruteForceAnalysis,
12   generateStrings
13 } = require("../helpers/palindromeOracle");
14
15 describe("palindromeService", () => {
16   test("находит наибольший нечетный палиндром", () => {
17     const result = analyzeString("abacaba");
18
19     expect(result.longestPalindrome.value).to.equal("abacaba");
20     expect(result.longestPalindrome.length).to.equal(7);
21     expect(result.palindromeCount).to.equal(12);
22   });
23 }

```

```

24 test("находит наибольший четный палиндром", () => {
25     const result = analyzeString("abba");
26
27     expect(result.longestPalindrome.value).to.equal("abba");
28     expect(result.longestPalindrome.length).to.equal(4);
29 });
30
31 test("возвращает служебные поля сравнения и подстроку для неполного пал
индрома", () => {
32     const result = analyzeString("cabbaq");
33
34     expect(result.algorithm).to.equal("manacher");
35     expect(result.longestPalindrome).to.deep.equal({
36         value: "abba",
37         start: 1,
38         end: 4,
39         length: 4
40     });
41     expect(result.comparison).to.deep.equal({
42         referenceAlgorithm: "center-expansion",
43         matches: true
44     });
45 });
46
47 test("формирует сопоставимый отчет для расширения от центра", () => {
48     const result = analyzeWithCenterExpansion("abba");
49
50     expect(result.algorithm).to.equal("center-expansion");
51     expect(result.longestPalindrome.value).to.equal("abba");
52     expect(result.palindromeCount).to.equal(6);
53 });
54
55 test("корректно восстанавливает массивы радиусов для эталонного алгорит
ма", () => {
56     const { oddRadii, evenRadii } = collectCenterExpansionRadii("abba");
57
58     expect(oddRadii).to.deep.equal([1, 1, 1, 1]);
59     expect(evenRadii).to.deep.equal([0, 0, 2, 0]);
60 });
61
62 test("корректно восстанавливает палиндром по радиусам", () => {
63     const best = findLongestPalindrome("abba", [1, 1, 1, 1], [0, 0, 2,
        0]);
64
65     expect(best).to.deep.equal({
66         value: "abba",
67         start: 0,
68         end: 3,
69         length: 4
70     });
71 });
72

```

```

73 test("формирует рекомендацию о полной симметрии", () => {
74     const recommendation = buildRecommendation("level", {
75         value: "level",
76         length: 5
77     });
78
79     expect(recommendation).to.equal("Строка целиком является палиндромом.
80         ");
81 });
82 test("формирует рекомендацию о крупном палиндромном фрагменте", () => {
83     const recommendation = buildRecommendation("abaxyz", {
84         value: "abax",
85         length: 4
86     });
87
88     expect(recommendation).to.equal("Строка содержит крупный палиндромный
89         фрагмент.");
90 });
91 test("формирует рекомендацию о коротких палиндромах", () => {
92     const recommendation = buildRecommendation("abcd", {
93         value: "a",
94         length: 1
95     });
96
97     expect(recommendation).to.equal("В строке преобладают только короткие
98         палиндромы.");
99 });
100 test("относит палиндром длины 2 к коротким палиндромам", () => {
101     const recommendation = buildRecommendation("aabb", {
102         value: "aa",
103         length: 2
104     });
105
106     expect(recommendation).to.equal("В строке преобладают только короткие
107         палиндромы.");
108 });
109 test("формирует рекомендацию о локальных палиндромных областях", () =>
110     {
111         const recommendation = buildRecommendation("abaxyzz", {
112             value: "aba",
113             length: 3
114         });
115
116         expect(recommendation).to.equal("Строка содержит несколько локальных
117             палиндромных областей.");
118     });
119 test("суммирует радиусы палиндромов", () => {

```

```

119     expect(sum([1, 2, 3])).to.equal(6);
120     expect(sum([0, 0, 0])).to.equal(0);
121   });
122
123   test("выбрасывает ошибку для пустой строки", () => {
124     expect(() => analyzeString("")).to.throw("Входная строка должна быть
125       непустой");
126   });
127
128   test("совпадает с независимым oracle на полном наборе коротких строк",
129     () => {
130     for (const input of generateStrings(6)) {
131       const result = analyzeString(input);
132       const oracle = bruteForceAnalysis(input);
133
134       expect(result.oddRadii).to.deep.equal(oracle.oddRadii);
135       expect(result.evenRadii).to.deep.equal(oracle.evenRadii);
136       expect(result.longestPalindrome).to.deep.equal(oracle.
137         longestPalindrome);
138       expect(result.palindromeCount).to.equal(oracle.palindromeCount);
139     }
140   });
141
142   test("выбрасывает ошибку при рассогласовании эталонного алгоритма по зн
143     ачению", () => {
144     expect(() =>
145       analyzeWithCenterExpansion("abba", {
146         referenceExpansion: () => ({ value: "zzzz", length: 4 })
147       })
148     ).to.throw("Эталонный анализ расширением от центра сформирован некорр
149       ектно");
150   });
151
152   test("выбрасывает ошибку при рассогласовании эталонного алгоритма по дл
153     ине", () => {
154     expect(() =>
155       analyzeWithCenterExpansion("abba", {
156         referenceExpansion: () => ({ value: "abba", length: 3 })
157       })
158     ).to.throw("Эталонный анализ расширением от центра сформирован некорр
159       ектно");
160   });
161
162   test("выбрасывает ошибку при рассогласовании алгоритмов по значению пал
163     индрома", () => {
164     expect(() =>
165       analyzeString("abba", {
166         referenceAnalyzer: () => ({
167           longestPalindrome: { value: "bb", length: 4 },
168           palindromeCount: 6
169         })
170     ).to.throw("Эталонный анализ расширением от центра сформирован некорр
171       ектно");
172   });

```

```

163     ).toThrow("Результаты алгоритмов не совпадают");
164   });
165
166   test("выбрасывает ошибку при рассогласовании алгоритмов по длине палиндрома", () => {
167     expect(() =>
168       analyzeString("abba", {
169         referenceAnalyzer: () => ({
170           longestPalindrome: { value: "abba", length: 3 },
171           palindromeCount: 6
172         })
173       )
174     ).toThrow("Результаты алгоритмов не совпадают");
175   });
176
177   test("выбрасывает ошибку при рассогласовании алгоритмов по числу палиндромов", () => {
178     expect(() =>
179       analyzeString("abba", {
180         referenceAnalyzer: () => ({
181           longestPalindrome: { value: "abba", length: 4 },
182           palindromeCount: 5
183         })
184       )
185     ).toThrow("Результаты алгоритмов не совпадают");
186   });
187 });

```

jsonReader.test.js

```

1  const fs = require("fs");
2  const os = require("os");
3  const path = require("path");
4  const { Readable } = require("stream");
5  const sinon = require("sinon");
6  const { extractInput, loadInputFromJson, trimBuffer } = require("../src/io/jsonReader");
7
8  describe("jsonReader", () => {
9    afterEach(() => {
10      sinon.restore();
11    });
12
13    test("корректно читает входную строку из JSON", async () => {
14      const file = path.join(os.tmpdir(), "palindrome-input.json");
15      fs.writeFileSync(file, JSON.stringify({ input: "level" }), "utf8");
16
17      await expect(loadInputFromJson(file)).resolves.toBe("level");
18    });
19
20    test("выбрасывает ошибку при отсутствии поля input", async () => {

```

```

21     const file = path.join(os.tmpdir(), "palindrome-invalid.json");
22     fs.writeFileSync(file, JSON.stringify({ value: "level" }), "utf8");
23
24     await expect(loadInputFromJson(file)).rejects.toThrow(
25       "JSON должен содержать непустое поле input"
26     );
27   });
28
29   test("выбрасывает ошибку, если файл не найден", async () => {
30     await expect(loadInputFromJson("missing-file.json")).rejects.toThrow(
31       "Файл не найден");
32   });
33
34   test("выбрасывает ошибку при пустом поле input", async () => {
35     const file = path.join(os.tmpdir(), "palindrome-empty-input.json");
36     fs.writeFileSync(file, '{ "input": "" }', "utf8");
37
38     await expect(loadInputFromJson(file)).rejects.toThrow(
39       "JSON должен содержать непустое поле input"
40     );
41   });
42
43   test("использует stub для имитации потоковой загрузки файла", async ()
44     => {
45     sinon.stub(fs.promises, "access").resolves();
46     sinon.stub(fs, "createReadStream").returns(Readable.from(['{ "input":
47       "racecar" }']));
48
49     await expect(loadInputFromJson("virtual.json")).resolves.toBe("
50       racecar");
51   });
52
53   test("корректно читает поле input даже при дополнительных полях", async
54     () => {
55     const file = path.join(os.tmpdir(), "palindrome-streamed.json");
56     fs.writeFileSync(
57       file,
58       JSON.stringify({
59         meta: { author: "student", tag: "practice" },
60         input: "abacaba",
61         options: { ignoreCase: false }
62       }),
63       "utf8"
64     );
65
66     await expect(loadInputFromJson(file)).resolves.toBe("abacaba");
67   });
68
69   test("извлекает поле input с экранированными символами", () => {
70     expect(extractInput('{ "input": "aba\\\\"caba" }')).toBe('aba"caba');
71   });

```

```

68   test("извлекает поле input при наличии пробелов перед двоеточием", ()
      => {
69     expect(extractInput('{ "input" : "level" }')).toBe("level");
70   });
71
72   test("обрезает буфер до позиции ключа input при наличии служебного преф
      икса", () => {
73     const trimmed = trimBuffer('service-prefix::{"input":"abba"}');
74
75     expect(trimmed).toBe('{"input":"abba"}');
76   });
77
78   test("ограничивает размер рабочего буфера до хвоста потока", () => {
79     const tail = trimBuffer("x".repeat(5000));
80
81     expect(tail.length).toBe(4096);
82   });
83 });

```

resultWriter.test.js

```

1  const fs = require("fs");
2  const os = require("os");
3  const path = require("path");
4  const sinon = require("sinon");
5  const { expect } = require("chai");
6  const {
7    formatResultLines,
8    printResult,
9    writeResultToFile
10 } = require("../src/io/resultWriter");
11
12 describe("resultWriter", () => {
13   afterEach(() => {
14     sinon.restore();
15   });
16
17   const sampleResult = {
18     input: "abba",
19     oddRadii: [1, 1, 1, 1],
20     evenRadii: [0, 0, 2, 0],
21     longestPalindrome: { value: "abba", start: 0, end: 3, length: 4 },
22     palindromeCount: 6,
23     recommendation: "Строка содержит крупный палиндромный фрагмент."
24   };
25
26   test("формирует человекочитаемые строки отчета", () => {
27     const lines = formatResultLines(sampleResult);
28
29     expect(lines).to.deep.equal([
30       "Исходная строка: abba",

```



```

31     "Нечетные радиусы: [1, 1, 1, 1]",
32     "Четные радиусы: [0, 0, 2, 0]",
33     "Наибольший палиндром: abba",
34     "Длина: 4",
35     "Количество палиндромных подстрок: 6",
36     "Рекомендация: Строка содержит крупный палиндромный фрагмент."
37   });
38 });
39
40 test("печатает результат в консоль", () => {
41   const logger = sinon.spy();
42   printResult(sampleResult, logger);
43
44   expect(logger.callCount).to.equal(7);
45   expect(logger.calledWithExactly("Исходная строка: abba")).to.equal(
46     true);
47   expect(logger.calledWithExactly("Рекомендация: Строка содержит крупны
48     й палиндромный фрагмент.")).to.equal(true);
49 });
50
51 test("сохраняет результат в JSON-файл через stub", () => {
52   const file = path.join(os.tmpdir(), "palindrome-result.json");
53   const writeStub = sinon.stub(fs, "writeFileSync");
54
55   writeResultToFile(sampleResult, file);
56
57   expect(writeStub.calledOnce).to.equal(true);
58   expect(writeStub.firstCall.args[0]).to.equal(file);
59   expect(JSON.parse(writeStub.firstCall.args[1])).to.deep.equal(
60     sampleResult);
61   expect(writeStub.firstCall.args[2]).to.equal("utf8");
62 });
63 });

```

consoleUi.test.js

```

1 const sinon = require("sinon");
2 const { expect } = require("chai");
3 const { ConsoleUi } = require("../src/ui/consoleUi");
4 const {
5   assumeTrue,
6   assumeFileExists,
7   withAssumptions
8 } = require("../helpers/assume");
9
10 describe("consoleUi", () => {
11   afterEach(() => {
12     sinon.restore();
13   });
14
15   test("печатает меню", () => {

```

```

16     const logger = sinon.spy();
17     const ui = new ConsoleUi({
18         reader: { question: async () => "0", close: async () => {} },
19         logger
20     });
21
22     ui.printMenu();
23
24     expect(logger.calledWithMatch("Анализ палиндромных подстрок")).to.
25         equal(true);
26     expect(logger.calledWithMatch("Запустить алгоритм Манакера")).to.
27         equal(true);
28     expect(logger.calledWithMatch("Запустить расширение от центра")).to.
29         equal(true);
30 });
31
32 test("ensureInput выбрасывает ошибку, если строка не задана", () => {
33     const ui = new ConsoleUi({
34         reader: { question: async () => "0", close: async () => {} },
35         logger: () => {}
36     });
37
38     expect(() => ui.ensureInput()).to.throw("Сначала необходимо задать ст
39         року");
40 });
41
42 test("ensureResult выбрасывает ошибку, если результат не рассчитан", ()
43     => {
44     const ui = new ConsoleUi({
45         reader: { question: async () => "0", close: async () => {} },
46         logger: () => {}
47     });
48
49     expect(() => ui.ensureResult()).to.throw("Сначала необходимо выполнит
50         ь анализ");
51 });
52
53 test("обрабатывает команду запуска алгоритма Манакера и выхода", async
54     () => {
55     const questions = ["3", "0"];
56     const logger = sinon.spy();
57     const resultPrinter = sinon.spy();
58     const analyzer = sinon.stub().returns({ input: "abba" });
59     const ui = new ConsoleUi({
60         reader: {
61             question: async () => questions.shift(),
62             close: async () => {}
63         },
64         analyzer,
65         resultPrinter,
66         logger
67     });

```

```

61     ui.currentInput = "abba";
62
63     await ui.run();
64
65     expect(analyzer.calledOnceWithExactly("abba")).to.equal(true);
66     expect(resultPrinter.calledOnce).to.equal(true);
67 });
68
69 test("обрабатывает команду запуска эталонного алгоритма", async () => {
70     const questions = ["4", "0"];
71     const logger = sinon.spy();
72     const resultPrinter = sinon.spy();
73     const referenceAnalyzer = sinon.stub().returns({ input: "abba" });
74     const ui = new ConsoleUi({
75         reader: {
76             question: async () => questions.shift(),
77             close: async () => {}
78         },
79         analyzer: sinon.stub(),
80         referenceAnalyzer,
81         resultPrinter,
82         logger
83     });
84     ui.currentInput = "abba";
85
86     await ui.run();
87
88     expect(referenceAnalyzer.calledOnceWithExactly("abba")).to.equal(true);
89     expect(resultPrinter.calledOnce).to.equal(true);
90 });
91
92 test("обрабатывает ручной ввод и сбрасывает прошлый результат", async
93     () => {
94     const reader = {
95         question: async () => "manual palindrome",
96         close: async () => {}
97     };
98     const ui = new ConsoleUi({
99         reader,
100         logger: () => {}
101     });
102     ui.lastResult = { stale: true };
103
104     await ui.handleCommand("1");
105
106     expect(ui.currentInput).to.equal("manual palindrome");
107     expect(ui.lastResult).to.equal(null);
108 });
109
110 test("обрабатывает загрузку из JSON и сбрасывает прошлый результат",
111     async () => {

```

```

110     const reader = {
111       question: async () => "input.json",
112       close: async () => {}
113     };
114     const loader = sinon.stub().resolves("loaded palindrome");
115     const ui = new ConsoleUi({
116       reader,
117       loader,
118       logger: () => {}
119     });
120     ui.lastResult = { stale: true };
121
122     await ui.handleCommand("2");
123
124     expect(loader.calledOnceWithExactly("input.json")).to.equal(true);
125     expect(ui.currentInput).to.equal("loaded palindrome");
126     expect(ui.lastResult).to.equal(null);
127   });
128
129   test("показывает последний результат по отдельной команде", async () =>
130     {
131     const resultPrinter = sinon.spy();
132     const ui = new ConsoleUi({
133       reader: { question: async () => "0", close: async () => {} },
134       resultPrinter,
135       logger: () => {}
136     });
137     ui.lastResult = { input: "abba" };
138
139     await ui.handleCommand("5");
140
141     expect(resultPrinter.calledOnce).to.equal(true);
142   });
143
144   test("показывает справку и сообщает о неизвестной команде", async () =>
145     {
146     const logger = sinon.spy();
147     const ui = new ConsoleUi({
148       reader: { question: async () => "0", close: async () => {} },
149       logger
150     });
151
152     await ui.handleCommand("7");
153     await ui.handleCommand("99");
154
155     expect(logger.calledWithMatch('Формат входного файла')).to.equal(true);
156     expect(logger.calledWithMatch("Неизвестная команда.")).to.equal(true);
157   });

```

```

157 test("не завершает программу аварийно при ошибке загрузки", async () =>
158   {
159     const questions = ["2", "broken.json", "0"];
160     const logger = sinon.spy();
161     const ui = new ConsoleUi({
162       reader: {
163         question: async () => questions.shift(),
164         close: async () => {}
165       },
166       loader: async () => {
167         throw new Error("Файл поврежден");
168       },
169       logger
170     });
171     await ui.run();
172
173     expect(logger.calledWithMatch("Ошибка: Файл поврежден")).to.equal(
174       true);
175   });
176 test("использует два assumption-style метода для проверки окружения",
177   async () => {
178     const helperPath = require.resolve("./helpers/assume");
179
180     await withAssumptions(async () => {
181       assumeTrue(typeof helperPath === "string" && helperPath.length > 0,
182         "Путь к helper не определен");
183       assumeFileExists(helperPath);
184
185       const logger = sinon.spy();
186       const ui = new ConsoleUi({
187         reader: { question: async () => "0", close: async () => {} },
188         logger
189       });
190
191       ui.printHelp();
192
193       expect(logger.calledThrice).to.equal(true);
194     });
195   });
196 test("использует mock для контроля записи результата", async () => {
197   const reader = {
198     question: async () => "report.json",
199     close: async () => {}
200   };
201   const writerHolder = {
202     writeResult() {}
203   };
204   const writerMock = sinon.mock(writerHolder);

```

```

204     writerMock.expects("writeResult").once().withExactArgs({ input: "abba
      " }, "report.json");
205
206     const ui = new ConsoleUi({
207         reader,
208         logger: () => {},
209         resultWriter: (...args) => writerHolder.writeResult(...args)
210     });
211     ui.lastResult = { input: "abba" };
212
213     await ui.handleCommand("6");
214
215     writerMock.verify();
216 });
217 });

```

tests/helpers/assume.js

```

1  const fs = require("fs");
2
3  class AssumptionError extends Error {
4      constructor(message) {
5          super(message);
6          this.name = "AssumptionError";
7      }
8  }
9
10 function assumeTrue(condition, message) {
11     if (!condition) {
12         throw new AssumptionError(message || "Предусловие теста не выполнено"
13             );
14     }
15 }
16
17 function assumeFileExists(path) {
18     assumeTrue(fs.existsSync(path), 'Ожидаемый файл не найден: ${path}');
19 }
20
21 async function withAssumptions(callback) {
22     try {
23         await callback();
24     } catch (error) {
25         if (error instanceof AssumptionError) {
26             return;
27         }
28         throw error;
29     }
30 }
31
32 module.exports = {

```

```
33 | AssumptionError ,  
34 | assumeTrue ,  
35 | assumeFileExists ,  
36 | withAssumptions  
37 |};
```